

AutoQuant: An Auditable Expert-System Framework for Execution-Constrained Auto-Tuning in Cryptocurrency Perpetual Futures

Kaihong Deng

*School of Modern Information Industry, Artificial Intelligence Program, Guangzhou
College of Commerce, Guangzhou, 510555, China*

Abstract

Backtests of cryptocurrency perpetual futures are sensitive to execution timing, funding alignment, trading costs, and reuse of evaluation windows during parameter search. In high-friction markets, attractive results may therefore reflect hidden implementation choices as much as signal quality. Using BTC/USDT, ETH/USDT, SOL/USDT, and AVAX/USDT perpetual contracts, this study examines whether an auditable execution-aware configuration-selection pipeline can reduce performance overestimation and expose parameter fragility more clearly than naive one-stage tuning.

This paper proposes AutoQuant, an expert-system-style decision-support framework for configuration selection. AutoQuant encodes strict execution timing, funding visibility, cost realism, and feasibility constraints as explicit rules; combines Bayesian search with two-stage screening across windows and cost scenarios; and exports deterministic artifacts with accounting-invariant checks for traceability. The resulting governance protocol selects and documents configurations within a pre-specified signal family under strict semantics.

Empirically, fee-only and zero-cost backtests materially inflate apparent performance relative to fully costed runs with funding and slippage. Two-stage screening does not guarantee higher returns; in the BTC anchor case and several replications, it more often surfaces lower-drawdown or less extreme alternatives under identical strict semantics. Same-budget optimizer

Email address: 202312100202@xs.gcc.edu.cn (Kaihong Deng)

comparison, module and screening-policy ablations, funding-rule diagnostics, inferential checks, cross-asset replications, and third-party replay checks position AutoQuant as auditable validation infrastructure for configuration selection under explicit execution and cost assumptions. The experiments use small-account simulations under linear costs and exclude market impact and institutional capacity constraints.

Keywords: expert systems, decision support, auditability, cryptocurrency derivatives, perpetual futures, backtesting realism, robustness screening

1. Introduction

Motivation and background.

The proliferation of cryptocurrency derivatives has fundamentally reshaped the digital asset landscape, with perpetual futures contracts emerging as a dominant instrument for both retail and institutional participants. Traded 24/7 on global exchanges, instruments such as the BTC/USDT perpetual exhibit high volatility, complex non-linear dynamics, and periodic liquidity crises. The market’s microstructure introduces unique challenges for quantitative strategy development, most notably through its intricate cost structure. Unlike traditional equity markets, a strategy’s profitability is co-determined by a mix of bilateral trading fees, periodic funding payments that anchor the contract price to the spot index, execution slippage, and stringent constraints on leverage and notional position sizes. The confluence of these factors makes the journey from a research backtest to live trading exceptionally fragile. From an expert-systems and decision-support perspective, the reliability of backtests in such markets directly affects how organizations build, validate, and govern algorithmic trading and risk-control components that may be embedded in digital-asset products, automated trading platforms, and DeFi-style protocols.

Within this high-stakes environment, the processes of backtesting and parameter tuning are prone to significant bias. Common methodological pitfalls frequently lead to substantial overestimation of performance. These include **look-ahead bias**, which arises from the improper use of future information, and **unrealistic cost assumptions**, where critical factors such as funding payments, slippage, and leverage constraints are ignored or oversimplified. Furthermore, many strategies rely on manual, *ad hoc* parameter tuning, yielding configurations that appear profitable in-sample but fail

to generalize to out-of-sample data or to the unforgiving conditions of live markets. This persistent gap between research and reality motivates the development of an implementation-oriented framework designed explicitly to bridge the divide between promising backtests and deployable strategies for cryptocurrency perpetuals.

Scope and interpretation. The empirical scope is deliberately bounded. AutoQuant is positioned as a configuration-selection and traceability protocol for a pre-specified signal family, with evidence from four liquid perpetual contracts (BTC/USDT, ETH/USDT, SOL/USDT, and AVAX/USDT). The cross-asset results serve as portability checks for the validation protocol rather than broad market validation.

Paper organization. The paper proceeds as follows. Section 2 presents the core methodology of the execution-centric auto-tuning and double-screening framework. The related literature is then reviewed, followed by the data and market setting, system architecture and implementation, empirical evaluation, discussion of implications and limitations, and the final conclusion.

Reproducibility pointer and timestamp convention. All core evidence in this manuscript is generated from deterministic machine-readable artifacts produced by the pipeline. Appendix A lists minimal scripts and commands to regenerate the reported tables and figures, subject to the data-access constraints described there. All date ranges are expressed in UTC and refer to the *bar-open timestamp* used to index the OHLCV series. Under STRICT4H, signals are computed at bar close and, by design, take effect only from the subsequent bar.

This paper addresses a narrow but practically important question: under a strict 4-hour perpetual-futures protocol, does execution-aware, double-screened configuration selection reduce the performance inflation and parameter fragility that commonly arise under naive one-stage tuning and simplified costs? AutoQuant is designed to answer this question in an auditable way. Rather than treating execution timing, funding visibility, trading costs, and screening policy as informal implementation details, it turns them into explicit system rules whose effects can be inspected, stress-tested, and replayed. Empirically, the study is anchored on BTC/USDT and extends the same protocol to ETH/USDT, SOL/USDT, and AVAX/USDT as bounded portability checks.

From an expert-systems and decision-support perspective, this paper therefore positions AutoQuant as validation and governance infrastructure for execution-constrained configuration selection in cryptocurrency perpetual

futures. The system encodes strict execution semantics, funding alignment, cost realism, and feasibility constraints as inspectable policy, and exports deterministic artifacts and accounting-invariant checks to support traceable offline-to-live consistency. In high-friction perpetual markets, the central claim is narrower: credible backtest evidence requires configuration selection rules that are explicit enough to be inspected and replayed.

Contributions. The contributions are fourfold. First, this paper formulates configuration selection for frictional perpetual-futures strategies as an auditable expert-system-style decision-support problem in which execution semantics, funding visibility, trading costs, and feasibility constraints are encoded as explicit system rules rather than treated as informal backtest choices. Second, it implements a policy-driven inference pipeline that combines Bayesian search with a two-stage double-screening protocol under strict $t + 1$ execution semantics and explicit cost-mis-specification scenarios. Third, it introduces a traceability layer based on deterministic machine-readable artifacts, accounting-invariant checks, and offline-to-live consistency mechanisms, so that configuration selection can be replayed and traced end-to-end. Fourth, through a BTC/USDT anchor case together with cross-asset replications on ETH/USDT, SOL/USDT, and AVAX/USDT, it shows how the framework exposes cost-driven performance inflation, window fragility, and residual overfitting risk under a unified strict protocol.

Nomenclature and abbreviations. Throughout the manuscript, **STRICT4H** denotes the strict 4-hour execution protocol used in the case study; **OHLCV** denotes open, high, low, close, and volume bar data; **TPE** denotes the Tree-structured Parzen Estimator sampler used in Stage I; **WFE** denotes walk-forward evaluation; **CAGR** denotes compound annual growth rate; **MaxDD** denotes maximum drawdown; **DSR** denotes the deflated-Sharpe-style diagnostic used in the long-window analysis; and **PBO** denotes probability of backtest overfitting.

Limitations of existing practice.

While numerous open-source frameworks such as Backtrader [1], Zipline [2], Freqtrade [3], and FinRL [4] have democratized algorithmic trading research, they typically prioritize strategy prototyping over strict realism. In many equity or low-friction settings this is acceptable if the user configures the system carefully. The concern here is narrower: in high-friction cryptocurrency perpetual markets, simplified defaults can yield backtests that look attractive on paper but do not survive realistic cost and execution mod-

eling. In official repositories and user guides, critical choices about execution semantics and cost realism are usually left to the end user. As a result, no-look-ahead invariants may not be enforced architecturally, cost models are often reduced to fees with only partial support for funding and leverage constraints, and parameter search is easily decoupled from realistic risk controls.

These frameworks are not treated as flawed in general. The point is that without strict guardrails, common usage patterns in perpetual-futures studies can create a substantial illusion of profitability. Appendix A illustrates the implementation burden with both a minimal Backtrader raw-return semantics check and a stronger Backtrader event-loop replay over the executed STRICT4H exposure path. These checks require bespoke data plumbing, timestamp alignment, explicit funding inputs, consistent fee/slippage/leverage constraints, deterministic export of per-bar return and cost components, and full-chain accounting invariants. They are therefore useful portability evidence for adapter-based traceability in generic trading frameworks.

Why generic research frameworks are not enough in this setting.

The comparison with common research frameworks is not intended to argue that such frameworks are unusable. The point is narrower and more operational: in high-friction perpetual-futures markets, credible evaluation requires that execution delay, funding visibility, leverage limits, notional caps, and per-bar cost accounting be treated as enforceable semantics rather than as optional user discipline. The practical contribution of AutoQuant is therefore to reduce the implementation gap between a backtest that can be run and one whose semantics can be checked and replayed.

A more fundamental limitation is that much prior academic and practitioner work remains **alpha-centric**. The main emphasis is usually on new predictive signals or factors, while the validation pipeline is treated as a secondary concern. Comparatively less attention is paid to the systematic, automated, and robust generation and validation of the *parameters* that govern these signals. The machinery that transforms a raw strategy idea into a vetted, tradable candidate therefore remains under-explored.

Table 1: **AutoQuant versus common trading research frameworks (feature matrix)**. Entries indicate whether a feature is *native*, *configurable*, or absent ($\times$). This classification reflects the capabilities described in official documentation and common usage patterns, and is not intended as an exhaustive survey of all possible extensions.

Framework	Funding	$t + 1$ semantics	Scenario-grid stress	Bayesian tuning	Full-chain invariants	Guard overlay	Perp-specific constraints
AutoQuant (this paper)	native	native	native	native	native	native	native
Backtrader [1]	configurable	configurable	$\times$	configurable	$\times$	configurable	configurable
Zipline [2]	$\times$	configurable	$\times$	configurable	$\times$	$\times$	$\times$
Freqtrade [3]	$\times$	configurable	$\times$	configurable	$\times$	configurable	$\times$
FinRL [4]	$\times$	configurable	$\times$	configurable	$\times$	configurable	$\times$

2. Methodology: Execution-Centric Auto-Tuning and Double Screening

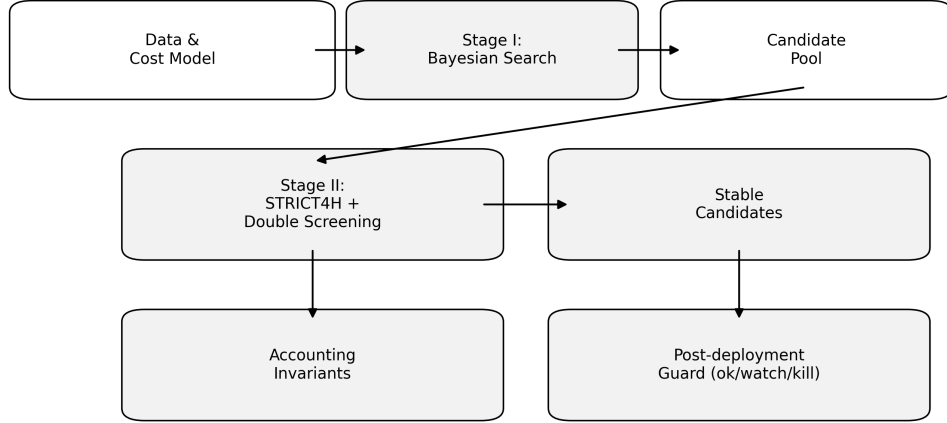
This section presents AutoQuant as an execution-aware decision-support protocol for robust configuration selection. The emphasis is not on inventing a new signal family, but on making the assumptions that govern evaluation and selection explicit, enforceable, and reproducible. Concretely, the methodology combines a bounded Stage I search, a Stage II double-screening procedure under strict semantics, and a deployment-oriented supervision layer that together turn fragile backtest choices into inspectable system policy.

Expert-system decomposition. For readers from the expert-systems and applied AI community, AutoQuant should be understood not as a symbolic reasoner for market prediction, but as a rule-explicit decision-support framework for robust configuration selection. Its knowledge base comprises enforceable operational rules: strict $t + 1$ execution, no-look-ahead funding alignment, explicit cost and constraint profiles, and ex ante screening thresholds. Its inference engine is the search-and-screen pipeline that evaluates candidate configurations under those rules. Its explanation and governance layer exports deterministic artifacts, reconciliation checks, and replay-style supervision signals so that the resulting recommendation can be independently traced and verified. In this sense, the contribution is methodological and architectural: AutoQuant turns fragile backtest assumptions into inspectable system policy.

Figure 1 provides a high-level overview of the two-stage workflow and its traceability interfaces, including window splits, scenario grids, and stable-candidate selection.

Table 2: **ESWA-aligned decision-support mapping of AutoQuant components.**

ESWA component	AutoQuant realization	Decision-support / traceability output
Knowledge base	STRICT4H execution semantics; funding alignment; cost/constraint profiles; screening policies	Declarative rules that prevent look-ahead and enforce feasibility; explicit cost-scenario grid and window semantics.
Inference engine	Stage I Bayesian optimization (TPE) + Stage II policy-driven double-screening	Stable configuration recommendation with robustness diagnostics across windows and cost-mis-specification scenarios.
Explanation interface	Deterministic machine-readable exports; accounting invariants; full-chain consistency checks	Traceable artifacts for table/figure re-generation and backtest-to-live consistency checks.
Governance overlay	Guard triggers + invariant monitoring	Run-time supervision signals and reconciliation deltas to detect drift and implementation inconsistencies.



STRICT4H enforces $T+1$ execution and no-look-ahead funding decisions.

Figure 1: **AutoQuant methodology flowchart.** Stage I Bayesian search produces candidates that undergo Stage II double-screening; stable sets are supervised by a guard overlay. STRICT4H enforces $t+1$ execution and no-look-ahead funding alignment.

2.1. Algorithmic Summary (Pseudo-code)

To make the end-to-end procedure directly reproducible from the text, the pipeline is summarized as a minimal interface and pseudo-code. Inputs are (i) a time-aligned 4-hour OHLCV series, (ii) an 8-hour funding-rate series aligned to the STRICT4H grid, (iii) a fixed cost profile and a predefined cost-scenario grid, (iv) a window split (Training/Validation/Long-horizon/Blind hold-out), and (v) a Stage I search budget N_{opt} . Outputs are the selected configuration, the backtest logs, and the exported machine-readable artifacts described in Appendix A.

```
# Stage I: Bayesian search (train net return)
history = []
for i in 1..N_opt:
    theta_i ~ TPE(history)
    r_net = STRICT4H_backtest(
        data=Training, theta=theta_i, costs=baseline_costs
    )
    score_i = annualized_return(r_net)
    history.append((theta_i, score_i))
Theta_pool = top-K candidates by score_i

# Stage II: double-screening (validation for selection only)
for theta in Theta_pool:
    for scenario in cost_grid:
        r_net = STRICT4H_backtest(
            data=Train+Val, theta=theta, costs=scenario
        )
        compute monthly return, drawdown, turnover
select theta_star via filters and scenario metrics

# Stress tests (blind hold-out; not used for selection)
evaluate theta_star on Long-horizon and Blind hold-out windows
export tabular summaries
```

In the implementation, the “export tabular summaries” step is backed by deterministic machine-readable outputs, including configuration `json`, per-bar ledger `csv`, robustness summaries, and invariant-check files that support manuscript table/figure reconciliation and backtest-to-replay checks when the companion data artifacts are available, as summarized in Appendix A.

2.2. The Signal-Modular Parameter Space

The framework deliberately treats the underlying alpha-generation logic as a black-box mapping

$$f : \mathcal{I}_t \times \Theta \rightarrow S_t,$$

where $\mathcal{I}_t$ denotes the information set available at time t , Θ the hyperparameter space, and $S_t \in \{-1, 0, 1\}$ the target position or trading signal. In the STRICT4H case study, the core momentum signal is derived from the normalized divergence between fast and slow exponential moving averages of the 4-hour close, compressed into $[0, 1]$ via a smooth tanh transform and combined with auxiliary mean-reversion and breakout channels. The details of this construction are implementation-specific; the focus here is on the structure and constraints of Θ rather than on the full functional form of the alpha engine itself.

While the AutoQuant architecture is signal-agnostic, the specific hyperparameter search space Θ in this manuscript is optimized for trend-following and mean-reversion families on cryptocurrency perpetual futures. Extending the framework to high-frequency market-making or arbitrage strategies would require redefining Θ and the associated cost-model primitives, although the validation pipeline would remain conceptually unchanged. Importantly, AutoQuant does not attempt to invent f automatically; the novelty lies in enforcing strict semantics and providing an auditable selection-and-screening pipeline for $\theta \in \Theta$.

$$m_t(\theta_{\text{momentum}}) = \frac{1}{2} \left[1 + \tanh \left(\frac{\text{EMA}_{\text{fast},t} - \text{EMA}_{\text{slow},t}}{\sigma(\theta_{\text{momentum}}) \text{EMA}_{\text{slow},t}} \right) \right]. \quad (1)$$

$$C_t = w_{\text{mom}} \cdot m_t + (1 - w_{\text{mom}}) \cdot a_t. \quad (2)$$

$$\text{Signal}_t = \begin{cases} +1, & C_t \geq \tau_t^{\text{long}}, \\ -1, & C_t \leq 1 - \tau_t^{\text{short}}, \\ 0, & \text{otherwise.} \end{cases} \quad (3)$$

Instead of enumerating implementation-specific variable names, the manuscript defines the hyperparameter space Θ in terms of a small number of interpretable blocks. The momentum and mean-reversion block controls the EMA lookback lengths, volatility-scaling factors, and Bollinger-band widths and tail quantiles that determine how quickly the strategy reacts to

trends or reversals. The risk-control block specifies minimum holding times, cooldown periods after exits, ATR-based stop-loss and take-profit multipliers, and several exposure-smoothing and maximum-exposure constraints. The Bayesian optimization stage explores Θ within fixed bounds chosen to respect realistic leverage, exposure, and cost constraints.

To mitigate adverse funding costs, the framework introduces a soft funding-bias mechanism that raises entry thresholds in periods of expensive carry. Let fr_t denote the 8-hour funding rate aligned to the 4-hour grid via carry-forward, and let τ_{base} be a baseline long-entry threshold. For long positions, the effective threshold is defined as

$$\tau_{\text{long}}(t) = \tau_{\text{base}} + \kappa \cdot \max(|fr_t| - \vartheta_{\text{bias}}/10000, 0), \quad (4)$$

where $\vartheta_{\text{bias}} \geq 0$ is an activation threshold and $\kappa \geq 0$ controls how quickly the threshold increases as funding moves deeper into the tails.

The optimization process in Stage I is guided by a single scalar objective dominated by cost-adjusted annualized return. Formally,

$$\max_{\theta \in \Theta} J(\theta), \quad J(\theta) := r_{\text{ann}}(\theta), \quad (5)$$

where $r_{\text{ann}}(\theta)$ is the annualized net return computed from the STRICT4H net-return series under the Stage I window and the baseline cost profile.

Performance metrics. The primary performance metric, annualized return r_{ann} , is based on geometric compounding of the 4-hour strategy returns over the entire backtest period. Let R_{tot} denote the total compounded return over N 4-hour bars, and let AF be the annualization factor (set to 2190):

$$r_{\text{ann}} = (1 + R_{\text{tot}})^{AF/N} - 1. \quad (6)$$

Unless otherwise stated, annualized return is reported as CAGR in decimal units. Maximum drawdown (MaxDD) is reported as the peak-to-trough equity decline over the window.

2.3. Stage I: Bayesian Auto-Tuning under Realistic Costs

The first stage of the framework employs Bayesian optimization to efficiently search the parameter space. The implementation uses the Tree-structured Parzen Estimator algorithm [5], as implemented in Optuna [6], to guide the search.

Why TPE Relative to Grid Search, Evolutionary Methods, and GP-BO. TPE is selected because the search space Θ contains mixed continuous/discrete pa-

Table 3: **Minimal STRICT4H hyperparameter summary (Θ) with default bounds and fallbacks.** Ranges correspond to the default Stage I Optuna search space. “Default” denotes the fallback value used when a parameter is omitted.

Block	Parameter	Range (default)	Constraint / interpretation
Momentum	<code>ema_fast</code>	6–32 (12)	EMA span in 4h bars; must be shorter than <code>ema_slow</code> .
Momentum	<code>ema_slow</code>	20–96 (26)	EMA span in 4h bars; <code>ema_fast</code> < <code>ema_slow</code> .
Momentum	<code>ema_threshold</code>	0.0005–0.0060 (0.003)	Entry threshold on EMA spread (decimal units).
Mean-rev	<code>bb_period</code>	10–30 (20)	Bollinger lookback in 4h bars.
Mean-rev	<code>bb_dev</code>	1.0–2.5 (2.0)	Band width in standard deviations.
Risk	<code>min_hold_bars</code>	1–6 (1)	Minimum holding duration in 4h bars.
Risk	<code>cooldown_hours</code>	0–8 (0)	Cooldown after exits to reduce rapid flip-flopping.
Risk	<code>atr_period</code>	0–30 (0)	ATR period in 4h bars; 0 disables ATR-based exits.
Risk	<code>atr_k_sl</code> , <code>atr_k_tp</code>	0–3 / 0–5 (0)	ATR stop-loss / take-profit multipliers; 0 disables.
Risk	<code>max_exposure_abs</code>	0–5 (0)	Account-level exposure cap.
Funding	<code>funding_bias_thr_bps</code>	0–10 (0)	Soft threshold before raising entry thresholds under adverse funding.
Funding	<code>funding_bias_k_thr_per_bps</code>	0–0.005 (0)	Threshold slope under adverse funding.

rameters, conditional blocks, and hard feasibility constraints. In this setting, TPE is a practical and widely used sampler for conditional search spaces and performs well in compute-limited regimes. By contrast, grid search scales poorly with dimension; random search is a strong baseline under limited budgets; evolutionary methods can be effective but often require larger evaluation budgets and more careful constraint handling; and Gaussian-process Bayesian optimization is less convenient for high-dimensional conditional spaces. Importantly, AutoQuant is sampler-agnostic: any black-box optimizer, including genetic algorithms, differential evolution, or GP-based Bayesian optimization, can be substituted if it proposes θ and consumes scalar scores. Such a substitution would not change the strict execution/cost semantics or the Stage II screening policies. The present manuscript therefore justifies TPE as a practical implementation choice rather than as an algorithmic contribution. To assess sampler sensitivity, the empirical section also reports a same-budget benchmark against random search, DE, GA, and GP-based Bayesian optimization. That benchmark supports sampler substitutability under the same strict evaluator; in the reported run, GP-BO finds the strongest training-window candidate.

In the BTC/USDT STRICT4H case study reported in this manuscript, the Stage I search budget is fixed ex ante at $N_{\text{opt}} = 40$ TPE trials on the training window. Annualized net return is used as the Stage I scalar objective to keep the optimization problem simple and to avoid embedding additional risk preferences into the sampler. Risk and fragility controls are instead enforced in Stage II as explicit, auditable policy constraints. A key design choice is that Stage I and Stage II evaluations share the same STRICT4H execution and funding-alignment semantics; Stage I differs only in its role, while Stage II re-evaluates a fixed candidate pool under additional windows and cost-mis-specification scenarios.

Optimizer baseline. As a minimal baseline under the same STRICT training window and objective, the best-of- N curve of TPE is compared against random sampling [7] in Table 4. This baseline is used as a matched-budget diagnostic showing that the search component is not merely an ad hoc selection device. Broader optimizer diagnostics are reported in Section 6.

Compute budget disclosure. The dominant cost driver is the number of rigorous STRICT4H backtest evaluations. Stage I runs $N_{\text{opt}} \in \{40, 120\}$ TPE trials on the training window, with one STRICT4H backtest per trial. Stage II then re-evaluates a fixed candidate pool across a small cost-scenario grid and multiple window segments. In the reproducible BTC snapshot, the

Table 4: **Optimizer baseline under identical STRICT training window.** Best-of- N objective (annual). Random values are mean $\pm$ std over seeds.

Budget N	TPE best	Random best (mean $\pm$ sd)
10	0.0174	-1.0000 $\pm$ 0.0000
20	0.0362	-0.7832 $\pm$ 0.4848
40	0.1172	-0.7832 $\pm$ 0.4848
60	0.1334	-0.7832 $\pm$ 0.4848
80	0.1508	-0.6638 $\pm$ 0.4915
100	0.1590	-0.4600 $\pm$ 0.5163
120	0.1590	-0.3580 $\pm$ 0.6763

screening applies 9 cost scenarios to the top 40 candidates over the combined Training+Validation window (7,262 4-hour bars), totaling 360 strict backtest runs, plus a small number of long-horizon and blind hold-out stress-test evaluations. All experiments are CPU-only. On a representative workstation, end-to-end runtime is typically hours to under one day per asset under these budgets.

Supporting funding-rule diagnostic. To isolate whether funding-aware alignment rules matter beyond simply charging funding as a cost, STRICT execution and all costs are kept fixed, and only the funding-related gating and bias rules are disabled. Table 5 reports the resulting change in performance for a representative tuned configuration. The diagnostic tests whether funding awareness can change the decision path itself, not only the ex post PnL calculation. Because this single component diagnostic is not a broad module-ablation suite, the main empirical claims do not depend on it.

Table 5: **Supporting funding-rule diagnostic (GP-BO candidate).** Costs and STRICT execution are held fixed; only funding-related gating, bias, and carry rules are disabled. The table reports component-level sensitivity.

Variant	Train monthly geom	Train MaxDD	Val monthly geom	Val MaxDD
Full	0.218	0.192	0.000	1.000
NoFundingGates	0.219	0.192	0.000	1.000

2.4. Stage II: Rigorous 4h Backtest and Double Screening

The candidate parameter sets generated in Stage I then enter a rigorous double-screening stage. The goal is no longer to search for new configura-

tions, but to determine whether seemingly attractive candidates remain credible once they are subjected to strict execution semantics, explicit funding-visibility rules, and bounded stress scenarios. The Stage II engine is therefore designed both to remove common sources of backtest inflation and to keep the offline evaluation path aligned with live-style execution logic.

Strict $t+1$ execution. A signal generated at the close of 4-hour bar t , using only information available at that time, can affect position and profit-and-loss only from bar $t+1$ onward. This prohibits same-bar execution and closes an important source of look-ahead bias.

No-look-ahead funding model. Funding costs are applied bar by bar. Funding rates are treated as an external time series aligned to the 4-hour grid via carry-forward, and the backtest uses only the funding values visible at each bar timestamp, scaled to bar duration. The engine is not allowed to backfill from future funding updates.

To make the accounting explicit, let $S_t \in \{-1, 0, 1\}$ denote the directional signal computed at the close of bar t from $\mathcal{I}_t$. STRICT4H enforces a strict $t+1$ delay by applying an executed signed nominal exposure π_t during bar t , derived from S_{t-1} after ex-ante caps, smoothing, and other fixed risk controls. Given the close-to-close market return r_t^{mkt} in Eq. (8), the raw strategy return is $r_t^{\text{raw}} = \pi_t r_t^{\text{mkt}}$. Transaction costs are applied on exposure changes at bar boundaries, $\Delta\pi_t = \pi_t - \pi_{t-1}$, with fees and slippage proportional to $|\Delta\pi_t|$. Funding costs accrue on notional exposure π_t using the aligned 8-hour funding series scaled to 4-hour bars. The resulting per-bar net return is

$$r_t^{\text{net}} = r_t^{\text{raw}} - C_{\text{fee},t} - C_{\text{slip},t} - C_{\text{fund},t}. \quad (7)$$

All reported performance metrics in the empirical section are computed from the resulting $\{r_t^{\text{net}}\}$ series unless stated otherwise.

Stage II applies two validation streams to a fixed pool of Stage I candidates; there is no additional re-optimization in this phase.

Held-out window robustness. Each candidate is re-evaluated across multiple overlapping rolling windows of fixed length (2,000 bars in the BTC case study, stepped forward by 500 bars) that are held out from Stage I optimization but may be reused within Stage II. This differs from classical walk-forward optimization because the parameter set is not re-trained on each window; instead, the same fixed configuration is applied repeatedly to assess temporal stability.

Cost-scenario robustness. The same parameter sets are re-evaluated under a bounded set of cost-mis-specification profiles. For stable-candidate fil-

Table 6: **Core STRICT4H symbols, units, and accounting bases.** Unless stated otherwise, returns and costs are in decimal units; bps inputs are converted by dividing by 10,000.

Symbol	Units / range	Meaning
S_t	$\{-1, 0, 1\}$	Directional signal computed at the close of bar t .
π_t	unitless, bounded	Executed signed nominal exposure held during bar t , applied with a strict $t+1$ delay from S_{t-1} after fixed caps, smoothing, and other risk controls.
r_t^{mkt}	decimal	Close-to-close market return of the underlying over bar t .
$C_{\text{fee},t}, C_{\text{slip},t}$	decimal	Turnover-based trading costs applied at bar boundaries, proportional to $ \Delta\pi_t $; bps inputs apply to notional turnover.
$C_{\text{fund},t}$	decimal	Funding cost accrued on notional exposure π_t using the aligned 8-hour funding-rate series scaled to 4-hour bars.

tering, the framework uses a discrete sensitivity grid that varies taker fees and funding multipliers ($\text{taker_bps} \in \{3, 4, 6\}$ and $\text{fund_mult} \in \{0.5, 1.0, 1.5\}$), producing scenario-aggregated monthly and drawdown summaries for each candidate. This grid includes both optimistic and pessimistic perturbations around a conservative baseline to make cost sensitivity explicit. As an additional stress test on selected configurations, pessimistic-cost scaling scenarios are also reported that multiply transaction-related costs by fixed factors (for example, $2\times$ and $3\times$) together with a funding-off stress.

Stable-candidate selection then proceeds in three transparent steps. First, absolute robustness thresholds are applied to remove underperforming configurations. In the BTC STRICT4H experiments these ex-ante rules include a minimum average monthly return, a floor for the worst scenario-level monthly return, a ceiling on average maximum drawdown, and a cap on position-switch density. Second, the surviving candidates are ranked by average monthly return in descending order and average maximum drawdown in ascending order over the training-plus-validation regime. Third, the top K candidates are retained as the final pool of stable BTC/USDT STRICT4H configurations.

To make this policy transparent, Table 7 reports the exact thresholds used in the stable-candidate filter. In the replication code these rules are implemented as an ex-ante filter over scenario-aggregated performance summaries followed by a simple lexicographic ranking by mean performance and drawdown; they are not tuned post hoc on a per-experiment basis.

Table 7: Stable-candidate selection rules used in the BTC/USDT STRICT4H case study. After filtering, the top $K = 5$ candidates are retained by sorting `mean_monthly_true` descending and `maxDD_mean` ascending.

Statistic	Meaning	Threshold	Role
<code>mean_monthly_true</code>	mean monthly geometric return (across scenarios)	≥ 0.005	minimum performance
<code>min_monthly_true</code>	worst-case scenario-level monthly geometric return	≥ 0.0	downside floor
<code>maxDD_mean</code>	mean maximum drawdown (across scenarios)	≤ 0.30	drawdown cap
<code>switch_density_mean</code>	mean position-switch density (across scenarios)	≤ 0.12	turnover cap

Metric choice note. Stable candidates are ranked using monthly geometric mean, drawdown, and switching frequency because these metrics are directly interpretable and less distorted by extreme compounding in high-volatility regimes. Other stability metrics, such as Sharpe-, Sortino-, or Calmar-style summaries, can be substituted within the same policy layer, but a systematic comparison of screening metrics is outside the scope of the present study.

Stable-candidate statistics and aggregation. For a candidate parameter set and a cost scenario s from the predefined grid, the net return series $\{r_t^{\text{net}}\}$ is computed and resampled into calendar-month compounded returns $R_{m,s} = \prod_{t \in m} (1 + r_{t,s}^{\text{net}}) - 1$. The scenario-level monthly geometric mean is

$$g_s = \exp\left(\frac{1}{M} \sum_m \log(1 + \max(R_{m,s}, -0.999999))\right) - 1.$$

This value is reported as `monthly_true` = g_s . The stable-candidate filter aggregates these scenario-level statistics across the scenario grid:

$$\begin{aligned} \text{mean_monthly_true} &= \mathbb{E}_s[g_s], \\ \text{min_monthly_true} &= \min_s g_s, \\ \text{maxDD_mean} &= \mathbb{E}_s[\text{maxDD}_s]. \end{aligned}$$

Position-switch density is computed per scenario as

$$\text{switch_density}_s = \#\{\Delta\pi_t \neq 0\}/N,$$

and the reported summary is $\text{switch_density_mean} = \mathbb{E}_s[\text{switch_density}_s]$. These definitions match the exported robust-summary files referenced in Appendix A.

3. Related Literature

This section positions AutoQuant relative to three literature streams that are usually studied separately but interact tightly in frictional perpetual-futures research: realistic backtesting under market frictions, hyperparameter search and selection bias in quantitative trading, and deployment-oriented traceability and governance for decision-support systems. This organization makes clear that the contribution lies less in proposing a new predictive model than in connecting these strands into an auditable configuration-selection protocol.

3.1. Expert Systems, Decision Support, and Auditable Pipelines

AutoQuant is positioned as an *expert-system-like*, rule-based decision-support pipeline: it codifies non-negotiable semantics and constraints as a knowledge base, uses an inference procedure (Bayesian search plus policy-driven screening) to select robust configurations, and exports traceable artifacts (Section 2.1 and Table 2). This emphasis is aligned with decision-support objectives in operational fintech contexts, where governance depends on explicit measurement, execution, and control layers rather than on opaque “best backtest” reports [8, 9]. In this sense, AutoQuant is closer to an execution-aware, traceability-oriented AutoML workflow [10] than to a strategy-generation system: the signal family is fixed, and the contribution lies in making configuration selection reproducible and inspectable under strict $t+1$ semantics and cost realism.

3.2. Backtesting and Execution Biases in Cryptocurrency Markets

Realistic backtesting is difficult even in traditional assets [11], and the problem is amplified in cryptocurrency perpetual futures. The 24/7 trading cycle and extreme volatility make data alignment and timestamp integrity critical [12, 13]. Prior work highlights look-ahead bias, microstructure frictions, arbitrage constraints, and liquidity fragmentation as recurring sources

of inflated backtest evidence in crypto markets [14]. These findings motivate strict execution assumptions rather than post-hoc adjustments.

More specific to perpetual contracts, funding modeling remains a major source of error. Recent work on perpetual-futures pricing, basis risk, and market quality shows that contract design and funding mechanics can be first-order drivers of both profitability and liquidity, so backtests that omit or misalign funding can be materially misleading [15–17]. A complementary systems-oriented review of perpetual futures across centralized and decentralized exchanges likewise shows how exchange design and trader behavior shape those mechanics [18]. Recent evidence also documents strong linkages between spot and derivatives markets, including price discovery in Bitcoin futures [19, 20]. Empirical work on modeling and forecasting perpetual-futures prices for trading applications further underscores that data construction and contract settlement mechanics matter for inference [21]. This work builds on that literature by programmatically enforcing $t+1$ execution and no-look-ahead funding calculations in the backtest engine, moving from problem identification to a traceable implementation pattern.

3.3. Parameter Tuning and Bayesian Optimization in Finance

Most quantitative strategies are highly sensitive to parameter choices. Manual tuning and naive grid search are inefficient and can promote overfitting [22]. Bayesian optimization and related methods can navigate large search spaces more efficiently [23].

Random search provides a competitive and widely used baseline for hyperparameter optimization under limited evaluation budgets [7], and evolutionary computation methods (e.g., genetic algorithms, differential evolution, and CMA-ES) provide a complementary family of black-box optimizers with different trade-offs in constraint handling and sample efficiency [24–26].

In finance, Bayesian optimization has been applied to trading rules and portfolio allocation, but a common limitation is that optimization is often decoupled from realistic cost and risk modeling. Parameters that appear strong in a simplified search environment can later prove fragile under real-world frictions. This concern is closely related to classic data-snooping and multiple-testing problems: repeated search over strategy variants or hyperparameters can induce selection bias even when each backtest is implemented correctly [27–30]. The use of DSR/PBO-style diagnostics is therefore a robustness check and complements rather than replaces strict execution and

cost semantics [31, 32]. The contribution here is to integrate Bayesian optimization with realistic costs from Stage I onward while keeping execution semantics fixed during Stage II screening.

3.4. Risk Management, Guards, and Full-Chain Consistency

A robust quantitative trading system requires more than a profitable signal; it also requires risk management and validation layers that remain consistent from research to live deployment. Risk-based overlays, or “guards,” that monitor realized performance and can reduce or halt trading are a familiar part of institutional practice. Recent work on decentralized finance likewise treats protocol-level risk controls and auditable accounting as first-class design objectives [33], reinforcing the view that infrastructure and monitoring are integral to decision support and model governance.

A parallel concern is the “full-chain consistency” between the backtest environment and the live execution engine. Divergences in data handling, cost calculation, or order logic can invalidate backtest results and lead to unexpected live performance [11]. This work synthesizes these two concerns by proposing a framework that not only includes a post-deployment guard but also enforces a set of accounting invariants across the entire backtest-to-live chain. This provides a formal, traceable link between theoretical research and practical implementation, a critical but often overlooked aspect of financial data science.

4. Data and Market Setting

This section outlines the financial instruments, data sources, and market assumptions that form the foundation of the empirical analysis. The study places a strong emphasis on data integrity and the realistic modeling of trading costs, since both are prerequisites for robust backtesting and transparent evaluation.

4.1. Instruments and Data Sources

The primary instrument for this study is the BTC/USDT perpetual futures contract, with parallel replications on ETH/USDT, SOL/USDT, and AVAX/USDT perpetual contracts using the same STRICT4H family and evaluation scripts. The experiments use 4-hour OHLCV bars sourced from major centralized-exchange perpetual-futures endpoints, with prices and volumes constructed from exchange-level candle fields and cleaned using the

procedures described below. In the anonymized replication package, the BTC/USDT anchor series is implemented from Binance, Bybit, and OKX perpetual data, and the ETH/USDT, SOL/USDT, and AVAX/USDT replications are generated through the same venue-recorded download and cleaning scripts for the corresponding perpetual contracts. The cleaned series used in each experiment is a single timestamp-aligned perpetual-futures OHLCV series per asset rather than a volume-weighted cross-venue synthetic price. When more than one venue snapshot is available, the replication scripts select a single continuous per-asset venue stream after continuity and integrity checks rather than merging prices across venues; the exact local venue/source path is recorded in the regeneration configuration. The replication materials provide regeneration scripts and local processed-data paths, while raw exchange files are not redistributed. The dataset is partitioned into a Core Sample (2019-09-08–2021-12-31, $N = 5,069$ 4h returns) used for descriptive statistics and core cost-accounting experiments, and an Extended Long-Horizon Sample (2019-09-08–2025-10-14, $N = 13,368$ 4h returns) used for long-horizon diagnostics and robustness stress tests.

Crucially, the dataset includes a synchronized time series of funding rates extracted at 8-hour intervals, aligned to the 4-hour time axis as a step function carried forward between funding timestamps. The fallback is activated only when a realized funding timestamp is missing or unavailable for the relevant historical slice; in that case, the conservative constant funding-rate assumption from the same trading-cost profile is used and the carry-forward, no-backfill convention is kept unchanged.

For ETH/USDT, SOL/USDT, and AVAX/USDT, the study uses the same construction convention: one cleaned 4-hour OHLCV series and one corresponding 8-hour funding-rate series per asset, drawn from the venue-specific perpetual-futures records retained in the reproducible snapshot. Data availability differs by asset: the ETH series spans 2021-12-31–2025-11-24, while the SOL/AVAX series span 2021-01-01–2025-12-16. They are evaluated under the same strict execution semantics and a conservative cost family, using asset-specific fee/slippage baselines together with the Stage II cost-sensitivity grid.

In this manuscript, the label *core* refers specifically to the BTC/USDT 4-hour segment used for the naive-versus-rigorous cost and constraint stress test. Separately, the BTC/USDT windowed experiments use explicit training/validation splits that extend through 2023-01-01, and long-horizon diagnostics extend through 2025-10-14.

Table 8: Data summary for BTC/USDT 4-hour returns.

Sample	Returns (4h)	Mean	Std. dev.	Min	Max
Core (2019-09-08–2021-12-31)	5,069	0.00043	0.01615	-0.21046	0.14734
Long (2019-09-08–2025-10-14)	13,368	0.00026	0.01309	-0.21046	0.14734

Throughout the paper, date ranges are interpreted as half-open intervals [start, end), so adjacent windows do not share bars. The following terminology is used for BTC/USDT 4-hour windows, with analogous windows for the ETH/SOL/AVAX replications:

- *Training window* (2019-09-08–2021-01-01): used exclusively for Stage I Bayesian optimization.
- *Validation window* (2021-01-01–2023-01-01): used within Stage II double-screening for robustness checks and selection, but not for Stage I candidate generation.
- *Long-horizon robustness window* (2019-09-08–2025-10-14): used for long-run diagnostics and cost-stress tests, not for parameter selection.
- *Blind post-ETF window* (2024-01-01–2025-10-14): held out from tuning and selection and used solely for post-hoc robustness assessment.

All Stage II thresholds and filters are fixed *ex ante* prior to evaluation on the long-horizon or blind hold-out windows, and hyperparameters, thresholds, or selection rules are not modified after observing hold-out outcomes.

Returns are computed as simple 4-hour arithmetic close-to-close returns,

$$r_t^{\text{mkt}} = \frac{P_t^{\text{close}}}{P_{t-1}^{\text{close}}} - 1 \quad (8)$$

where P_t^{close} denotes the 4-hour close price. Under STRICT4H execution semantics, the position applied to r_t^{mkt} is lagged by one bar, so that the raw strategy return at bar t depends only on information available by the close of bar $t - 1$.

Table 9: Mapping between evaluation stages and BTC/USDT 4-hour windows in the AutoQuant pipeline.

Stage / component	Primary role	Windows used	Key settings
Stage I (Bayesian search)	Parameter generation	Training	TPE; $N_{\text{opt}} = 40$
Stage II (double-screening)	Robustness screening	Training + validation	$K = 5$; WFE win=2000, step=500
Long-horizon diagnostics	DSR and cost stress	Long-horizon robustness	cost stress $2\times, 3\times$
Blind evaluation	Post-ETF robustness	Blind post-ETF	post-hoc only

4.2. Data Cleaning and Validation

The policy for handling OHLCV data gaps is conservative and designed to prevent the introduction of artificial price data. The system does not interpolate or fill missing OHLCV bars. Funding-rate timestamps are treated as a separate external series: when available, they are aligned to the 4-hour grid using carry-forward between funding updates with no backfilling from future updates; when unavailable, the system falls back to a conservative constant assumption from the cost profile under the same carry-forward semantics.

4.3. Data Construction Protocol (4-hour OHLCV)

To make the BTC/USDT 4-hour series traceable, the study adopts a simple and explicit construction protocol: use exchange-provided perpetual-futures candle fields as the primary price/volume source; represent each bar by its bar-open timestamp in UTC; enforce basic OHLCV sanity constraints; sort and de-duplicate by timestamp; run a strict continuity check at the target frequency; and construct the STRICT4H funding series separately as an external input with an explicit fallback when realized funding is unavailable.

In the unified codebase used for this manuscript, the data-fetch and merge logic is implemented in lightweight scripts that support direct inspection. The replication package includes scripts for fetching full-history K-lines, aggregating 1-hour bars into 4-hour OHLCV, validating continuity and data integrity, and generating or backfilling the aligned funding series described above.

5. System Architecture and Implementation

This section summarizes the implementation choices that are most relevant to the traceability of the empirical results under strict execution and cost modeling. The full AutoQuant codebase contains additional production components, including deployment utilities, exchange adapters, and monitoring dashboards. This manuscript focuses on the design choices that bear directly on research validity: (i) reuse of the same STRICT4H signal and cost engine across offline evaluation and live-style replay; (ii) explicit, reconcilable accounting of returns and costs; and (iii) a post-deployment guard that is intentionally separated from the optimization objective.

5.1. *Three Evaluation Stages: Data, Optimization, and Out-of-Sample Validation*

The workflow is organized into three stages. First, *data and environment validation* performs the integrity checks in Section 4.2 and fixes a cost profile and constraint set that remain unchanged during optimization and screening. Second, *optimization and screening* implements the two-stage methodology: Stage I generates candidates on a single training window under the realistic cost model, while Stage II screens candidates across held-out windows and cost-stress scenarios using the STRICT4H engine. Third, *live-style replay and risk control* applies the same STRICT4H logic on a rolling window of recent data to produce a target exposure and optionally overlays a guard for post-deployment supervision. In this paper, Stage (iii) is treated as a traceability pattern rather than as evidence of long-horizon live performance.

5.2. *The Live Engine and Executor*

The live engine is implemented as a thin wrapper around the same core STRICT4H strategy and cost logic used offline. On each update cycle it recomputes the target exposure from a bounded sliding window of recent 4-hour data and applies the same strict $t+1$ semantics described in Section 2.4. The executor then maps that target exposure to exchange orders while enforcing the same leverage and notional caps used in the backtest engine and logging fills and cash-flow components, including fees, slippage, funding, and PnL, for later reconciliation.

5.3. Full-Chain Invariants and Guard Infrastructure

A set of full-chain accounting invariants is defined: mathematical identities that must hold, within a small tolerance, across the entire lifecycle of a strategy instance. For any given parameter set, key metrics such as cumulative exposure, total fees, total slippage, net funding costs, and total PnL must reconcile between the offline backtest, live-style replay, and execution logs produced by the same code path when such logs are available.

Concretely, for each bar t , the system records the raw strategy return r_t^{raw} and cost components $C_{\text{fee},t}$, $C_{\text{slip},t}$, and $C_{\text{fund},t}$, and defines the net return

$$r_t^{\text{net}} = r_t^{\text{raw}} - C_{\text{fee},t} - C_{\text{slip},t} - C_{\text{fund},t}. \quad (9)$$

Full-chain consistency checks verify that these components reconcile across offline backtests, live-style replay, and execution logs produced by the same code path.

The repository-level deployment-bridge evidence is a strict- $t+1$ code-path parity check between the rigorous backtest engine and the live-style inference wrapper. Across three independent SOL/USDT 1-hour parameter snapshots over 2,000 bars each, the exported signal and exposure series matched exactly. This is reported as internal consistency evidence for the offline-to-live bridge rather than as evidence of long-horizon live performance.

Table 10: **Repository-level strict- $t+1$ parity snapshots (rigorous backtest vs replay path).** Values are intended as code-path consistency evidence, not as a live-performance claim.

Snapshot	Asset / freq	Bars	Direction match	$\max \Delta S $	$\max \Delta \pi $
Trial 37	SOL/USDT 1h	2000	1.0	0	0
Trial 71	SOL/USDT 1h	2000	1.0	0	0
Trial 82	SOL/USDT 1h	2000	1.0	0	0

A guard provides a post-deployment risk overlay that monitors rolling performance and risk metrics computed from execution logs and can issue an *ok/watch/kill* decision. A “kill” decision triggers position flattening and can optionally disable further trading until manual intervention. Because the guard responds to realized PnL and risk states rather than to predictive signals, it is treated as a safety layer that is conceptually separate from the Stage I/II optimization objectives. This paper reports only limited offline and replay-style consistency evidence and therefore interprets the guard component as a traceability pattern rather than as a live-performance claim.

5.4. Computational complexity and deployment feasibility

To make the practical footprint of AutoQuant explicit, the analysis distinguishes the computational burden of Stage I search, Stage II screening, and online deployment-style replay. Let T denote the number of bars in the evaluation window, N_{opt} the Stage I search budget, K the number of candidates retained for Stage II, and S the number of cost-stress scenarios. Under a fixed signal family, one strict backtest pass is linear in T up to small constant factors induced by the enabled signal channels and risk controls. Stage I therefore scales approximately as $\mathcal{O}(N_{\text{opt}}T)$, while the dominant Stage II screening cost scales approximately as $\mathcal{O}(KST)$ over the selected windows. In the reproducible case study these workloads remain practical on CPU-only hardware because the study deliberately uses bounded trial budgets, a small stress grid, and a fixed candidate pool rather than an open-ended nested search.

The implementation is also designed with deployment feasibility in mind. The same rigorous engine used for offline strict backtests is reused for live-style inference, so the offline and online paths do not rely on separate strategy logic. In the online wrapper, only a bounded recent history is retained, repeated polling on an unchanged last bar is short-circuited by a lightweight tail-bar fingerprint, and funding inputs are aligned under the same visibility rules used offline. This design does not constitute a claim of institutional-scale latency-sensitive execution, but it does show that the proposed framework is operationally feasible as a research-to-deployment decision-support pipeline for medium-frequency perpetual-futures strategies under explicit execution semantics.

6. Empirical Evaluation

This section evaluates AutoQuant as an execution-aware decision-support pipeline. The empirical analysis first compares optimizer behavior and component ablations under the same STRICT4H evaluator, then examines funding and screening sensitivity, and finally tests whether the validation protocol remains informative across additional assets, an auxiliary signal family, and third-party replay paths. Together, these experiments show how explicit execution rules, cost realism, and auditable screening change the interpretation of backtest results in frictional perpetual-futures markets.

Empirical roadmap. The empirical section is organized to answer six distinct questions: how several mainstream optimizers behave under the same strict-evaluation budget, which decision and screening modules materially affect selected candidates, whether funding-aware decision rules matter beyond ex post funding charges, whether selected configurations remain interpretable under inferential and policy-sensitivity checks, whether the validation protocol is portable across several liquid perpetual contracts under identical strict semantics, and whether a bounded auxiliary out-of-family strategy family survives the same screening rules.

6.1. *Experimental Setup*

The experiments use 4-hour OHLCV datasets for BTC/USDT, ETH/USDT, SOL/USDT, and AVAX/USDT perpetual contracts, built from the reproducible dataset described in Section 4. For the primary comparative experiment in Section 6.2, the core BTC sample is supplemented with the extended 4-hour series, and three distinct, non-overlapping windows are defined: an in-sample training window used exclusively for Stage I Bayesian optimization, a validation-and-selection window used within the Stage II double-screening protocol, and a blind test window used only for hold-out robustness assessment. For BTC, the 2023-01-01–2024-01-01 interval is intentionally left unused so that the validation-era selection window remains clearly separated from the post-ETF hold-out.

For ETH/USDT, the study design is mirrored with a training window (2021-12-31–2023-01-01), validation window (2023-01-01–2024-01-01), and post-2024 hold-out (2024-01-01–2025-11-24), using the same STRICT4H configuration family and evaluation scripts. For SOL/USDT and AVAX/USDT, the same training and validation definitions as ETH are used, and the post-2024 hold-out is extended through 2025-12-16.

Table 11: **Configuration naming used in the manuscript.** The text distinguishes *baseline* (a frozen reference from Stage I), *one-stage* (top-1 on training), and *selected two-stage* (stable-candidate screening). “Naive” and “standard” are diagnostic stress-test variants applied to a separate stress-test configuration.

Name in text	Definition	Used for selection?	Where referenced
Baseline STRICT4H	Frozen reference built from top-ranked Stage I parameters; evaluated across windows	No	Table 12, Fig. 2
One-stage (top-1)	Best Stage I trial by training-window objective	Yes (Stage I only)	Section 6.2
Selected two-stage	Stable candidate selected by ex-ante filter and scenario-grid aggregation	Yes (Stage II)	Section 6.2
Standard (fee-only)	Diagnostic backtest with taker fees only; same strict execution	No	Section 6.3
Naive (zero-cost)	Upper bound with fees/slippage/funding removed (and optional cap relaxation)	No	Section 6.3

To provide a point of comparison, the manuscript defines a Baseline Strategy configuration. This is not a toy example, but a realistic production-oriented baseline built from the top-ranked Stage I parameter sets for the BTC/USDT STRICT4H family and then frozen for all subsequent window and stress-test evaluations. It serves as a fixed reference configuration for inspection and ablation.

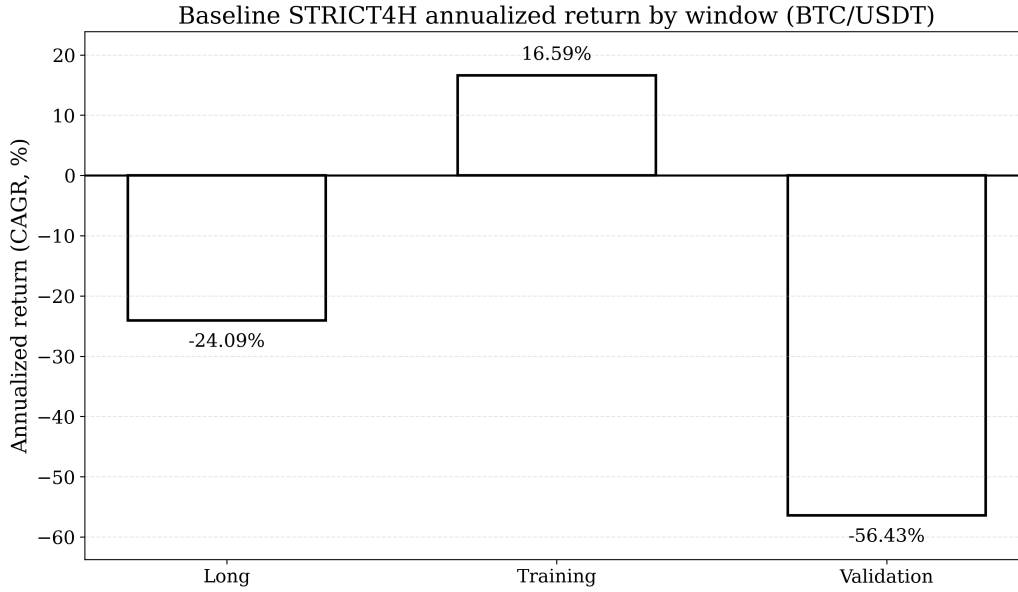


Figure 2: Baseline STRICT4H annualized return by window (BTC/USDT).

Table 12: Baseline STRICT4H performance by window (BTC/USDT; annualized return reported as CAGR, %; window definitions follow Section 4). Values are reported from the companion replication summaries described in Appendix A.

Window	Ann. (CAGR, %)	MaxDD	Sharpe	Trades
Long (2019-09-08–2025-10-14)	-24.09%	0.9536	-0.0738	4,814
Training (2019-09-08–2021-01-01)	16.59%	0.6041	0.5233	1,027
Validation (2021-01-01–2023-01-01)	-56.43%	0.9174	-0.5913	1,549

In keeping with the emphasis on interpretable and economically meaningful performance metrics, Table 12 reports only normalized statistics: annualized return, maximum drawdown, Sharpe ratio, and total trade count,

reported from STRICT4H machine-readable summaries in the replication package. The same outputs also contain raw total return and final-equity metrics, which are treated as internal accounting checks and are deliberately omitted from the main text because they are highly sensitive to sample length and the arbitrary choice of initial capital.

The Sharpe ratio is annualized from the 4-hour returns using the annualization implied by the 4-hour frequency and assumes a constant annual risk-free rate of 3%. Because 4-hour crypto returns are autocorrelated and heavy-tailed, Sharpe is treated as a descriptive statistic rather than as a parametric test. The analysis therefore focuses not only on absolute metric levels, but also on their sensitivity across time periods and cost assumptions.

Reading guide for compounded return magnitudes. Depending on the experiment, annualized return is reported either as CAGR in % or as CAGR in decimal units (e.g., $0.10 = 10\%$). In high-volatility regimes, compounded CAGR values can exceed 1.0 even under strict execution and explicit leverage caps. Large CAGRs are therefore treated as *diagnostic* signals of sensitivity to execution, window choice, and costs, and the interpretation places greater weight on drawdown, stability across windows, and cost-stress outcomes. None of the reported return magnitudes should be interpreted as statements about institutional-scale deployability, since non-linear market impact is not modeled.

6.2. One-Stage Tuning vs. Two-Stage Double Screening

The first experiment is designed to test the core hypothesis of the paper: that a simple one-stage parameter optimization is insufficient for identifying robust strategies. Two distinct approaches are compared.

One-Stage Tuning: Stage I Bayesian optimization is run exclusively on the training window to find the top-performing parameter set based solely on its in-sample, cost-adjusted annualized return.

Two-Stage Framework: The pool of high-performing candidates from Stage I is subjected to the full Stage II double-screening protocol, including out-of-sample rolling-window analysis and cost-mis-specification scenarios. The final selected parameter set is the one that ranks highest according to the stable-candidate criteria described in Section 2.4.

Table 13 makes the BTC/USDT anchor comparison explicit. Monthly geometric means are reported rather than headline CAGR in the main table because the latter can become visually dominant in high-volatility, small-sample regimes. The comparison is diagnostic: one-stage tuning can de-

liver stronger in-sample and validation return magnitudes, while the two-stage protocol makes the return–drawdown trade-off, turnover differences, and hold-out degradation visible under unchanged strict semantics. Buy-and-hold is also included as a simple external yardstick on the same OHLCV series.

Table 13: **Direct comparison: one-stage versus two-stage selection under identical STRICT4H execution and rigorous costs (BTC/USDT).** One-stage selects the single best Stage I trial on the training window. Two-stage selects a stable candidate via the ex-ante filter and cost-scenario aggregation described in Section 2.4. Validation is held out from Stage I but reused within Stage II screening; Post-2024 is a blind hold-out used only for robustness. The table reports monthly geometric net returns (decimal units), Sharpe, drawdown, and trade counts. Buy-and-hold is included as a simple external yardstick on the same OHLCV series.

Method	Window	Monthly geom	Sharpe	MaxDD	Trades
One-stage	Training	0.222	3.164	0.283	548
Two-stage	Training	0.145	2.388	0.265	485
Buy-hold	Training	0.067	1.530	0.563	—
One-stage	Validation	0.051	1.238	0.318	423
Two-stage	Validation	0.038	1.070	0.231	348
Buy-hold	Validation	-0.024	0.005	0.771	—
One-stage	Post-2024	0.023	0.845	0.220	259
Two-stage	Post-2024	0.016	0.664	0.196	223
Buy-hold	Post-2024	0.048	1.405	0.307	—

Interpretation relative to the BTC anchor case. The BTC anchor comparison highlights the selection trade-off directly. The two-stage protocol trails the one-stage winner on return magnitude and buy-and-hold in the post-2024 window, but under the same strict execution and cost semantics it selects a lower-turnover candidate with shallower drawdowns in validation and hold-out. This is the intended screening function: the protocol makes fragility and selection trade-offs visible before deployment.

Block-bootstrap check (validation window). As a minimal inferential supplement that is robust to time dependence, the analysis computes a moving block bootstrap [34] on the monthly return series for the validation segment, reporting a 95% interval for the difference in monthly geometric mean returns between the two-stage and one-stage procedures (two-stage minus one-stage).

Table 14: **Moving block bootstrap CI for monthly geometric mean differences (BTC/USDT validation).** Monthly geometric means are in decimal units; Δ is reported as two-stage minus one-stage.

Window	Δ monthly geom (dec.)	95% CI (dec.)	Block length (months)
Validation	-0.016	[-0.040, 0.008]	3

Threshold sensitivity. Because stable-candidate selection relies on ex-ante thresholds, the replication package reports a sensitivity scan in the replication package. Across reasonable variations of the monthly return floor, drawdown cap, and switch-density cap, the selected trial identity and its validation performance can change, highlighting that these thresholds function as *risk-control design choices* rather than as universal constants. The stable-candidate filter is therefore treated as a transparent, auditable policy rather than as an optimized meta-learner. In the $3 \times 3 \times 3$ grid (27 policy variants), the selected configuration toggles among three trial IDs (52, 87, 108; each selected 9 times), with the number of passing candidates ranging from 12 to 31. Across these policy variants, validation-window CAGR spans 0.332–1.058 (decimal), Sharpe spans 0.844–1.667, and MaxDD spans 0.209–0.386. Table 15 shows representative settings.

Table 15: **Representative threshold-sensitivity settings for stable-candidate selection (BTC/USDT).** Thresholds are ex-ante design choices; the table reports the resulting selected trial ID and its validation performance under rigorous costs. Annual return is CAGR in decimal units; MaxDD is a fraction in $[0, 1]$.

Monthly floor	MaxDD cap	Switch cap	Selected	Val CAGR (dec.)	Val Sharpe	Val MaxDD	Val trades
0.003	0.25	0.10	87	0.332	0.844	0.386	438
0.003	0.30	0.10	108	0.572	1.070	0.231	348
0.003	0.35	0.10	52	1.058	1.667	0.209	467

As an additional stress-test exercise, selected configurations are kept fixed and extended into the 2021-12-31–2025-10-14 BTC/USDT 4-hour period, including a blind post-ETF hold-out window (2024-01-01–2025-10-14). These extensions are treated as post-hoc robustness checks rather than as headline results, and they are not used for parameter selection or for the core claims of the paper.

6.3. Naive Backtest vs. Rigorous Backtest

The second experiment quantifies the degree of performance overestimation that results from extreme simplifications to the cost and constraint model. For this purpose, the study constructs a deliberately optimistic “Naive” configuration that holds the STRICT4H signal logic and $t+1$ execution semantics fixed but removes market frictions and relaxes capacity constraints. Relative to the rigorous configuration, the naive configuration (i) sets fees, slippage, and funding costs to zero; and (ii) relaxes leverage and notional caps so that the same signal path can be evaluated in a near-frictionless environment. This should be interpreted as an upper bound on friction- and constraint-related performance inflation in this setting, rather than as a representation of typical industry backtesting practice.

The analysis next isolates the impact of cost modeling itself. For this purpose, it takes a **separate STRICT4H configuration** used specifically for this stress test and evaluates it under a ladder of increasingly optimistic assumptions on the BTC/USDT core sample. The *rigorous* variant uses the full realistic trading-cost profile under strict $t+1$ execution. The *standard* variant keeps strict execution but includes fees only. The *naive* variant removes fees, slippage, and funding to form a zero-cost upper bound. Because cost-aware channels depend on the cost and funding inputs, the realized trade sequence and thus the trade count can differ across variants even when the hyperparameters are fixed.

Table 16: **Cost ablation ladder for the naive-versus-rigorous stress test (BTC/USDT core sample).** Fixed parameters; the realized trade sequence may differ across variants. Annual return is reported as CAGR in decimal units. Values may exceed 1.0 under compounding and should be interpreted as a diagnostic of cost sensitivity rather than as a scalability claim. Market impact is not modeled.

Variant	Ann. (CAGR, dec.)	Sharpe	MaxDD	Trades
Rigorous (fees+slippage+funding)	2.726	2.049	0.265	636
Standard (fee-only)	4.308	2.506	0.262	636
Naive (zero-cost, upper bound)	5.225	2.711	0.260	535

Table 16 quantifies how optimistic cost simplifications inflate reported performance for this fixed stress-test configuration. Even a fee-only “standard” backtest can overstate performance relative to a fully costed run that includes slippage and funding, and removing all costs provides an additional

optimistic upper bound. The table is reconciled to the companion cost-accounting ablation table in the replication materials, but it is used only as a diagnostic cost-sensitivity ladder. These results are treated as diagnostic baselines rather than as evidence of scalable profitability.

6.4. Robustness and Cross-Asset Replication

The final set of experiments evaluates the robustness of the parameter sets identified by the full two-stage framework under alternative conditions and across additional assets. First, the one-stage versus two-stage comparison is repeated on ETH/USDT using the same STRICT4H family and identical execution and cost semantics. Similar to the BTC anchor case, the two-stage selection makes the trade-offs explicit under unchanged semantics: the selected candidate uses far fewer trades and smaller drawdowns in two of the three windows, while the one-stage top-1 retains higher return magnitude. This supports pipeline-level portability of the screening mechanism across two liquid perpetual contracts.

Table 17: **Direct comparison: one-stage versus two-stage selection under identical STRICT4H execution and rigorous costs (ETH/USDT).** The table reports monthly geometric net returns (decimal units), Sharpe, drawdown, and trade counts.

Method	Window	Monthly geom (dec.)	Sharpe	MaxDD	Trades
One-stage (top-1)	Training	0.233	3.009	0.317	2112
Two-stage (selected)	Training	0.173	2.607	0.276	346
One-stage (top-1)	Validation	0.085	2.157	0.186	2012
Two-stage (selected)	Validation	0.046	1.368	0.201	457
One-stage (top-1)	Post-2024	0.151	2.590	0.381	3880
Two-stage (selected)	Post-2024	0.096	2.009	0.368	852

Second, to probe higher-friction mid-cap contracts, the same protocol is repeated on SOL/USDT and AVAX/USDT using stricter cost profiles but the same strict execution semantics. The results illustrate the intended role of double-screening as a *fragility filter*: the one-stage top-1 can exhibit extreme compounding and large drawdowns, whereas the two-stage selection more often shifts the choice toward lower-drawdown or less extreme configurations under stricter cost assumptions. The SOL/USDT case is a useful counterpoint because the two-stage candidate also improves several return metrics; this is treated as a market- and candidate-specific outcome rather

than a general rule. The extreme AVAX/USDT one-stage compounding is interpreted as a volatility and fragility diagnostic under linear costs.

Third, to test portability beyond a single contract, an ETH-trained STRICT4H aggressive configuration is evaluated on BTC/USDT over the overlapping window. Table 19 reports the key summary metrics under consistent rigorous trading-cost profiles, and Figure 3 visualizes annual return and maximum drawdown for the in-sample (ETH) and cross-asset (BTC) evaluations. The exercise is a diagnostic portability check under identical accounting and execution semantics.

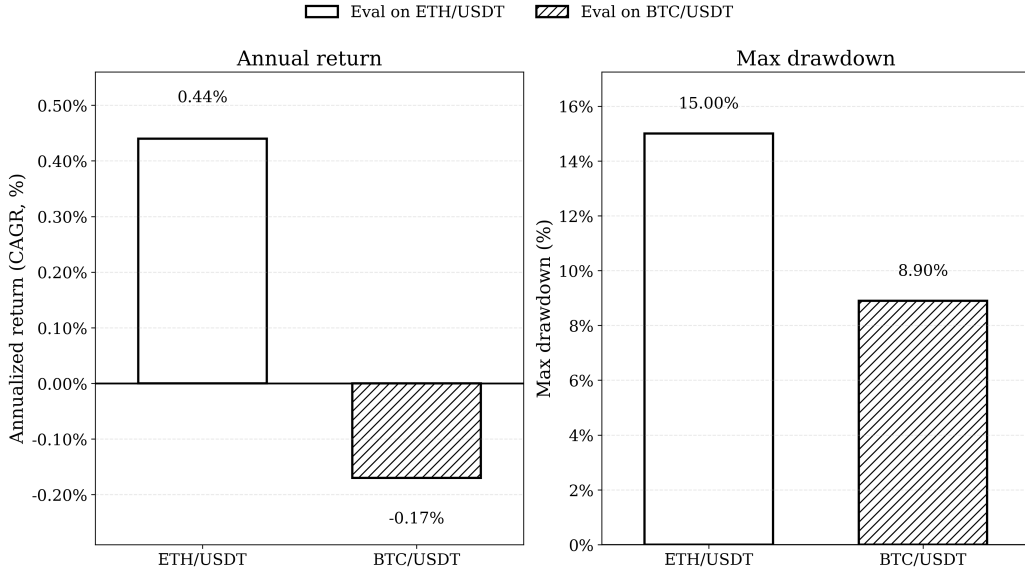


Figure 3: Cross-asset robustness of the ETH-trained STRICT4H aggressive configuration evaluated on ETH/USDT (in-sample) and BTC/USDT (cross-asset) over the overlapping data window.

6.5. Auxiliary Out-of-Family Check

To test whether the validation protocol remains informative outside the main STRICT4H family, the study includes one auxiliary BTC/USDT 4h experiment based on a long-only volatility-momentum family. The check preserves the same train, validation, and post-2024 window logic, applies the same strict execution and cost semantics, and evaluates candidates under the same Stage II threshold style used in the BTC case study. For this supplementary check, a deterministic 120-trial Stage I search budget is used,

Table 18: **Direct comparison on mid-cap contracts under stricter cost profiles (SOL/USDT and AVAX/USDT).** The table reports monthly geometric net returns (decimal units), Sharpe, drawdown, and trade counts.

Asset	Method	Window	Monthly geom (dec.)	Sharpe	MaxDD	Trades
SOL/USDT	One-stage (top-1)	Training	0.141	2.625	0.319	1567
SOL/USDT	Two-stage (selected)	Training	0.124	3.013	0.152	1605
SOL/USDT	One-stage (top-1)	Validation	0.227	3.398	0.239	1596
SOL/USDT	Two-stage (selected)	Validation	0.362	4.208	0.242	1653
SOL/USDT	One-stage (top-1)	Post-2024	0.066	1.747	0.243	3295
SOL/USDT	Two-stage (selected)	Post-2024	0.068	2.301	0.195	3450
AVAX/USDT	One-stage (top-1)	Training	0.479	3.239	0.795	2063
AVAX/USDT	Two-stage (selected)	Training	0.091	4.856	0.055	1739
AVAX/USDT	One-stage (top-1)	Validation	0.572	3.851	0.621	1908
AVAX/USDT	Two-stage (selected)	Validation	0.087	4.185	0.096	1118
AVAX/USDT	One-stage (top-1)	Post-2024	0.797	4.407	0.607	4158
AVAX/USDT	Two-stage (selected)	Post-2024	0.094	3.994	0.142	3663

Table 19: ETH STRICT4H cross-asset performance under consistent realistic trading-cost profiles for ETH and BTC over the overlapping data window (annual return reported as CAGR in %).

Strategy	Train	Eval	Ann. (%)	MaxDD	Trades
ETH STRICT4H aggressive v1	ETH	ETH	0.44%	0.150	3,100
ETH STRICT4H aggressive v1	ETH	BTC	-0.17%	0.089	3,403

and the top-30 candidates were screened over the same 3×3 taker/funding scenario grid. Under this larger auxiliary search, one candidate passed the pre-registered Stage II thresholds.

The auxiliary results clarify protocol behavior outside the main family in two ways. First, the one-stage top-1 candidate looked attractive in the training window but collapsed on validation, showing the same overfitting risk that motivated the double-screening protocol in the main BTC study. Second, one candidate did pass the Stage II acceptance thresholds under the cost-stress grid, but the resulting configuration remained modest: its validation mean monthly geometric return was 0.0062 and its post-2024 annual return was slightly negative. This auxiliary check gives bounded empirical evidence that the protocol can be applied beyond the main signal family.

To check whether the evaluator can accommodate optimizer choices beyond TPE, a bounded auxiliary optimizer check was also run on the same auxiliary volatility-momentum family. Random search, differential evolu-

Table 20: **Auxiliary BTC/USDT 4h volatility-momentum family check under the same strict screening logic.** The one-stage top-1 candidate is reported alongside the best Stage II candidate under the 120-trial follow-up search. Annual return is reported as CAGR in decimal units.

Candidate	Window	Ann. return	Monthly geom	MaxDD
One-stage top-1	Training	0.3046	0.0225	0.193
One-stage top-1	Validation	0.0000	0.0000	1.000
One-stage top-1	Post-2024	0.1469	0.0112	0.154
Best Stage II candidate	Training	0.0197	0.0016	0.102
Best Stage II candidate	Validation	0.0772	0.0062	0.061
Best Stage II candidate	Post-2024	-0.0052	-0.0004	0.100

tion (DE), a simple genetic algorithm (GA), and Gaussian-process Bayesian optimization (GP-BO) were each given 18 strict train-window evaluations and were then evaluated on the validation and post-2024 windows under the same cost profile. Table 21 shows that alternative black-box optimizers can be routed through the same STRICT4H evaluator. It also shows that several methods find train-window candidates whose validation behavior is weak or degenerate, which is precisely the kind of fragility the screening layer is designed to expose.

Table 21: **Bounded auxiliary optimizer check on the BTC/USDT 4h volatility-momentum family.** All methods use the same STRICT4H train-window objective and the same low evaluation budget. The table reports bounded substitutability evidence.

Method	Eval.	Best train	Val ann.	Val MaxDD	Post ann.	Post MaxDD
Random	18	0.1832	0.0241	0.0243	-0.0003	0.0944
DE	18	0.0992	-0.0006	0.1460	0.0688	0.0977
GA	18	0.1579	0.0000	1.0000	0.1890	0.2195
GP-BO	18	0.1352	0.0000	1.0000	0.2002	0.1904

6.6. Main-Family Optimizer Comparison and Component Diagnostics

The auxiliary optimizer check is complemented by a same-budget benchmark on the main BTC/USDT STRICT4H family. TPE, random search, DE, a simple GA, and GP-based Bayesian optimization were routed through the same shared parameter space, objective, cost profile, and 60-evaluation budget. Table 22 reports sampler-comparison evidence: in this bounded run,

GP-BO produced the strongest training candidate and the best post-2024 monthly geometric return among the five methods, while all methods’ best training candidates were weak on the 2021–2022 validation window. This is consistent with the paper’s core claim that optimizer output must be passed through strict screening rather than accepted at face value.

Table 22: **Main-family same-budget optimizer benchmark on the BTC/USDT STRICT4H training window.** All optimizers use the same shared main-family parameter space, objective, cost profile, and evaluation budget. Validation and post-2024 metrics evaluate the best training-window candidate found by each optimizer. Mon. denotes monthly geometric return, DD denotes maximum drawdown, and tr. denotes trades. The benchmark is interpreted as bounded evaluator-comparison evidence.

Method	Eval.	Invalid %	Train obj.	Val mon.	Val DD	Post mon.	Post DD	Post tr.
TPE	60	36.7	0.1870	0.0000	1.000	0.0159	0.545	437
Random	60	66.7	0.0748	0.0000	1.000	0.0166	0.385	364
DE	60	63.3	0.0450	0.0000	1.000	-0.0151	0.427	203
GA	60	28.3	0.0498	0.0000	1.000	-0.0003	0.356	2204
GP-BO	60	45.0	0.2184	0.0000	1.000	0.0400	0.291	867

Selected decision and execution modules were then disabled on the best main-family candidate returned by each optimizer. Table 23 reports the post-2024 effect on the GP-BO candidate and the median effect across the five optimizer-selected candidates. Disabling the ADX/regime gate substantially increased drawdown and turnover; disabling confirmation and turnover controls increased switching; disabling signal pre-averaging reduced post-2024 monthly return in the median candidate. Funding-rule removal was near-neutral in the median post-2024 comparison, although it changed the individual GP-BO candidate’s return and drawdown. The module ablation is therefore interpreted as component-level diagnostic evidence.

Finally, alternative Stage II policies were applied to the same valid top-20 main-family trial pool. Table 24 varies the rolling step size, threshold strictness, and cost-grid inclusion while keeping the candidate pool fixed. No candidate passed even under the relaxed threshold or the no-cost-grid variant, indicating that the same-budget optimizer candidates remain weak selections under the paper’s stability policy.

Table 23: **Module ablation on the main-family benchmark candidates.** The table shows the post-2024 effect on the GP-BO best training candidate and, for ablated variants, the median post-2024 change across the five optimizer-selected candidates. Mon. denotes monthly geometric return, DD denotes maximum drawdown, tr. denotes trades, and sw. denotes switch density. Positive ΔDD means larger drawdown. The experiment is a bounded component diagnostic, not a universal component ranking.

Variant	GP mon.	GP DD	GP tr.	Med. Δ mon.	Med. ΔDD	Med. Δ sw.
Full	0.0400	0.291	867	–	–	–
NoFunding	0.0490	0.231	867	+0.0000	+0.000	-0.0005
NoPreAvg	0.0129	0.181	660	-0.0197	-0.161	-0.0123
NoRegime	0.0256	0.848	3643	-0.0045	+0.443	+0.1283
NoConfirmTurn	0.0413	0.303	3567	+0.0025	+0.013	+0.1089
NoBoostStruct	0.0400	0.291	867	+0.0000	+0.000	+0.0000

Table 24: **Stage-II rolling-step and screening-threshold ablation.** Each policy screens the same valid main-family trial pool over the validation window. The base policy uses 2,000-bar rolling windows stepped by 500 bars and the ex-ante thresholds in Table 7.

Policy	Step	Scen.	Pass	Mean thr.	Min thr.	MaxDD cap	Switch cap	Top pass monthly
Base	500	9	0	0.0050	0.0000	0.30	0.12	–
Step 250	250	9	0	0.0050	0.0000	0.30	0.12	–
Step 1000	1000	9	0	0.0050	0.0000	0.30	0.12	–
Relaxed	500	9	0	0.0030	-0.0020	0.40	0.18	–
Strict	500	9	0	0.0075	0.0020	0.25	0.08	–
No cost grid	500	1	0	0.0050	0.0000	0.30	0.12	–

7. Discussion

This section interprets the empirical evidence as decision-support evidence for execution-aware configuration selection. The discussion focuses on what the experiments imply about execution realism, configuration fragility, residual overfitting risk, and the practical boundaries of deployment-oriented decision support in perpetual futures.

Interpretation of the empirical contribution. These experiments should therefore be read as evidence about the behavior of an execution-aware validation protocol. The contribution is not a new alpha engine; it is a disciplined way to show when apparent profitability depends on optimistic execution assumptions, fragile window selection, or insufficiently explicit governance of configuration choice.

Scope boundaries and limitations. This manuscript emphasizes five scope limits. First, the optimizer evidence is bounded: although the paper reports a same-budget comparison across TPE, random search, DE, GA, and GP-based Bayesian optimization, the result is a bounded single-study diagnostic rather than a universal optimizer ranking. Second, although the manuscript documents an explicit OHLCV and funding-construction protocol, exchange-specific data-construction choices can still affect perpetual-futures studies and remain a valid target for future data-pipeline checks. Third, the trading-cost model is linear in turnover; nonlinear market impact and venue-specific liquidity constraints are left to future execution models. Fourth, the core evidence remains restricted to a bounded strategy family and a small set of liquid perpetual contracts; the cross-asset results, module checks, and one auxiliary out-of-family check are portability and fragility diagnostics for the protocol. Fifth, although the implementation includes third-party replay checks, a local Freqtrade adapter backtest, deployment-oriented parity checks, and guard-based supervision, broader third-party integrations and production exchange deployment remain future work.

Protocol portability beyond one signal family. Architecturally, AutoQuant is signal-modular: the strict engine consumes a parameterized signal-and-risk specification, while the traceability protocol operates at the level of execution semantics, cost realism, screening policy, and artifact traceability. For this reason, the contribution is not limited to one specific momentum rule. At the implementation level, the codebase exposes multiple signal blocks, including

momentum, mean-reversion, breakout, volatility-momentum, structural, and factor-style components, under the same strict execution engine. Repository tests confirm the narrower point that the same runtime can ingest different parameter branches, including momentum, mean-reversion, and blended multi-leg specifications, without changing the underlying execution semantics. To probe this point empirically, a bounded BTC/USDT 4h auxiliary check using a long-only volatility-momentum family under the same window logic and Stage II screening rules was also run; the one-stage top-1 candidate collapsed on validation and one candidate passed the Stage II acceptance thresholds. This shows that the protocol can expose fragility outside the main signal family while keeping strategy-family generality as a future empirical question. Implementation-level checks therefore support protocol portability within the reported scope.

Residual selection-bias diagnostics. Even under strict execution semantics and explicit cost modeling, repeated search over candidate configurations leaves non-trivial multiple-testing risk. The manuscript therefore reports two selection-bias diagnostics from the reproducible BTC/USDT anchor snapshot. First, following the spirit of [31], a DSR-style check based on the long-window Sharpe and an effective trial-count range $N_{\text{total}} \in [360, 3240]$ is consistent with non-negligible search risk once repeated trials are taken into account. Second, a CSCV/PBO diagnostic [32] computed on the top-40 Stage I candidate pool yields $\text{PBO} = 0.586$ under an 8-segment design with 70 combinatorial splits. These diagnostics reinforce the manuscript’s emphasis on traceability and bias control under explicit execution rules.

Optimizer scope. Although TPE is justified as a practical sampler for conditional mixed search spaces, the optimizer evidence remains bounded. In addition to the original TPE-versus-random comparison, this paper includes a main-family same-budget comparison with DE, a simple GA, and GP-based Bayesian optimization. This broader check shows heterogeneous optimizer behavior: GP-BO is strongest in the reported training-window run, while the resulting candidates do not satisfy the validation screening policies. Accordingly, the contribution lies in the execution- and traceability-preserving configuration-selection pipeline, not in an optimizer-ranking result.

Data-construction disclosure. Section 4 documents the core OHLCV construction protocol, including UTC bar-open indexing, sanity filtering, deduplication, and continuity validation, and points to the scripts used to re-

build long-window 4-hour candles and funding baselines. Nevertheless, several exchange-specific data-construction choices can still matter in perpetual markets, including whether a venue’s OHLCV reflects last-trade, mark, or index pricing, cross-venue aggregation rules, and the handling of contract-specification changes. These details remain a natural target for future data-construction work.

Linear cost model and position scaling. The cost profile used in the STRICT4H experiments assumes an initial capital of 10,000 USDT, a maximum leverage of $5\times$, and a notional cap of 50,000 USDT. The reported annualized returns and Sharpe ratios should therefore be interpreted as describing small- to mid-sized positions in a frictional but non-impact world. The model does not include non-linear market impact or exchange-specific liquidity constraints, so the same percentage returns should not be extrapolated to institutional asset levels. Consistent with this view, very large final-equity figures that arise in long synthetic compounding runs are treated as accounting artifacts and deliberately omitted from the main tables. More broadly, the cost model is linear in turnover and does not capture non-linear impact; incorporating impact-aware execution models is outside the scope of the current manuscript.

Alpha logic constraints. While the framework is signal-modular at the architectural level, integrating highly complex, non-parametric machine-learning models would require additional engineering. The current auto-tuning pipeline is not designed to manage the separate training, validation, and GPU resource management loops required by such models, and the empirical evidence in this paper remains centered on a STRICT4H momentum-style family on BTC/USDT, ETH/USDT, SOL/USDT, and AVAX/USDT, plus one auxiliary BTC/USDT 4h volatility-momentum family check that yielded one Stage II-stable candidate in the larger follow-up search. Relatedly, automated *candidate generation* such as factor-library composition, template expansion, or symbolic regression is outside the scope of this manuscript. Such generators can still be evaluated as upstream modules under the same strict $t+1$ semantics, cost modeling, and multi-window screening used here.

8. Conclusion and Future Work

This paper introduced a systematic framework for the auto-tuning and multi-stage validation of quantitative strategies for cryptocurrency

perpetual futures. Using BTC/USDT as the anchor case and replications on ETH/USDT, SOL/USDT, and AVAX/USDT, the analysis showed that backtests which neglect realistic costs or permit subtle look-ahead effects can materially overstate strategy quality.

The primary contribution is a structured two-stage selection pipeline complemented by a post-deployment monitoring layer. Concretely, the framework combines Bayesian optimization under realistic cost models, a double-screening protocol using held-out and cost-stress scenarios, and an traceability/guard overlay. The result is an auditable decision-support and governance protocol for configuration selection under strict execution semantics.

Across the four contracts, realistic execution and costs materially change candidate rankings and reveal degradation on held-out windows. The non-BTC replications provide pipeline-level portability checks under identical strict semantics, and an auxiliary out-of-family BTC/USDT check further shows that the same protocol can reject unstable candidates outside the main signal family. These supplements broaden the evidence base and clarify the empirical scope of the framework. By enforcing full-chain accounting invariants and employing a guard mechanism, the framework provides a practical research-to-deployment pattern for this asset class. That is presented as a design objective supported by offline and replay-style consistency evidence.

From an expert-systems and decision-support perspective, the practical implication is that quantitative evidence in 24/7 derivatives markets should be delivered as an *auditable artifact*: a complete specification of timestamp conventions, execution semantics, cost and funding alignment, and window-reuse rules, together with deterministic scripts that regenerate the reported tables and figures. Appendix A therefore includes both full-chain consistency checks and minimal external semantics replays.

Managerial implications. For practitioners building algorithmic trading or risk-control pipelines in 24/7 derivatives markets, the evidence supports four governance practices: treat execution semantics and funding alignment as non-negotiable policies rather than optional backtest settings; pre-register window semantics and selection rules to avoid hidden reuse of validation segments; maintain deterministic per-run artifacts for independent inspection; and deploy lightweight guard overlays that monitor invariant violations and reconcile offline and live-style replays.

Finally, the numeric results reported in the tables and figures are accompanied by the unified codebase, anonymized replication materials, and derived summary artifacts described in Appendix A, subject to the data-

access and storage constraints discussed there. A central limitation remains that the present cost model is linear in turnover. Future work should incorporate nonlinear market impact, liquidity-dependent slippage, queue-position effects, and venue-specific execution constraints before drawing capacity or institutional deployment conclusions.

Appendix A. Reproducibility, Code, Data, and AI Tools

This paper places a strong emphasis on reproducibility and transparency. The experiments reported in this paper were prepared within a unified codebase that implements the backtest engine, Bayesian optimization pipeline, double-screening protocol, and live-guard logic described in the main text. For this manuscript, an anonymized replication package is maintained, built from the same codebase, that provides the core 4-hour datasets and/or scripts to regenerate them, together with configuration files, derived summary artifacts, and helper scripts needed to reproduce and check the reported backtests, validation procedures, and tables. This package can be shared with qualified researchers on reasonable request, subject to exchange data-licensing and storage constraints. The numeric results shown in the manuscript are reported from simulation outputs and derived machine-readable summaries so that the paper remains aligned with the underlying experiment logs; where only summary-level outputs are retained in the submission snapshot, the associated tables are treated as bounded empirical summaries rather than as fresh full reruns.

For the core tables and figures, the replication package includes deterministic scripts and summary artifacts that support regeneration or reconciliation of the machine-readable outputs used to produce the reported results, subject to the availability of the required market-data snapshots. A brief README lists the commands, expected output filenames, input-data requirements, and runtime notes. The package also includes a helper script that regenerates the figures used in this L^AT_EX submission.

The replication snapshot includes selected derived machine-readable artifacts and supporting reproducibility outputs that are used to reconcile the manuscript tables and figures. End-to-end regeneration scripts for the data-dependent tables, together with optional third-party semantics replays, are provided in the companion replication package; because full-chain logs can be large, this snapshot keeps selected derived artifacts used in the paper rather than every intermediate output. The auxiliary out-of-family check and

semantic/funding checks are directly machine-readable. The one-stage/two-stage validation rows and the cost-ablation ladder also have table consistency checks against companion summary artifacts. Several other large historical result tables remain traceable through companion summary artifacts rather than complete local rerun logs.

Execution-semantics reproducibility checks. The repository also includes regression tests and consistency checks for the strict $t+1$ execution convention, cost accounting, funding visibility, and selected carried-forward tables. These checks keep the manuscript’s empirical claims aligned with the submitted tables and reproducibility artifacts. Same-bar $t+0$ sensitivity results are reported below as execution-semantics diagnostics rather than selected-performance evidence.

Table A.25: **Minimal reproducibility artifacts exported by AutoQuant.** Each run exports machine-readable files that support table/figure regeneration or reconciliation and decompose net returns into raw-return and cost components. Exact filenames and schemas are documented in the replication README.

Artifact (format)	Contents (examples)	Reproducibility purpose
Configuration (json)	θ , cost profile, window spec, seeds	Reproducible run specification and provenance
Per-bar ledger (csv)	timestamp, signal S_t , exposure π_t , r_t^{mkt} , r_t^{raw} , $C_{\text{fee},t}$, $C_{\text{slip},t}$, $C_{\text{fund},t}$, r_t^{net}	Full reconstruction of metrics and cost decomposition
Robust summary (csv)	window/scenario aggregates (monthly geom, MaxDD, turnover)	Stage II screening inputs and sensitivity reporting
Invariant check (json/csv)	max $ \Delta S $, max $ \Delta \pi $, component diffs	Backtest-to-replay consistency checks

Execution-semantics diagnostic. As an additional minimal check reproducible from the replication snapshot, the appendix reports how performance changes when the same 4-hour strategy parameters are evaluated under strict $t+1$ execution versus a deliberately naive same-bar ($t+0$) convention. This semantic sensitivity check is separate from parameter selection. Values in Table A.26 are generated from the replication artifacts described above.

Batch semantics uplift (Stage I top- N). To avoid over-interpreting a single parameter setting, the appendix also reports a batch summary over the top- N Stage I trials. Each trial is re-evaluated on the same hold-out window under STRICT $t+1$ versus NAIVE $t+0$, with all other settings fixed. This

Table A.26: **Execution-semantics diagnostic (BTC/USDT 4h; 2022-01-01–2023-01-01)**. Fixed parameters; only the execution delay differs: strict $t+1$ versus same-bar $t+0$. Annual return is CAGR in decimal units; monthly geometric mean is in decimal units; MaxDD is a fraction in $[0, 1]$.

Semantics	Ann. CAGR	Monthly geom	Sharpe	MaxDD	Trades
STRICT ($t+1$)	-0.221	-0.021	-2.413	0.221	30
NAIVE ($t+0$)	-0.264	-0.025	-2.998	0.264	28

check asks whether same-bar execution *systematically* inflates performance in the snapshot. For BTC/USDT on 2022-01-01–2023-01-01 using the top $N = 30$ Stage I trials, the median uplift (NAIVE minus STRICT) in annualized return is negative and the fraction of trials with positive uplift is below 50%, indicating that naive same-bar execution is not a universal performance booster in this setting.

Table A.27: **Batch execution-semantics uplift (BTC/USDT 4h; 2022-01-01–2023-01-01; top $N = 30$ Stage I trials)**. Uplift is NAIVE($t+0$) minus STRICT($t+1$) in annualized return (CAGR, decimal units).

N	Median uplift	25th pct	75th pct	% uplift > 0
30	-0.018	-0.361	0.033	0.367

Third-party semantics replay (Backtrader). As a minimal external check of the strict close-to-close $t+1$ raw-return accounting, the same directional signal series is replayed in Backtrader under cheat-on-close execution, and the resulting per-bar raw-return series is compared to the vectorized STRICT4H construction. On BTC/USDT 4h over 2022-01-01–2023-01-01, the maximum absolute per-bar difference is on the order of 10^{-6} and the correlation is effectively 1.0, supporting the claim that the strict raw-return semantics are not an artifact of a single bespoke implementation.

Table A.28: **Backtrader raw-return replay check (BTC/USDT 4h; 2022-01-01–2023-01-01)**. Raw returns only; costs and funding are disabled for this semantics-only check.

Bars	Max $ \Delta $	RMSE	Corr.
2190	2.75×10^{-6}	2.94×10^{-7}	1.000

Third-party framework replay benchmark. For external framework comparison, a Backtrader adapter replay was also run on the main-family GP-BO candidate. The adapter uses Backtrader’s event loop to replay the executed STRICT4H exposure series and then applies the same fee, slippage, and funding ledger used by AutoQuant. On the post-2024 BTC/USDT window, the maximum absolute per-bar net-return difference was 5.44×10^{-5} , with RMSE 4.68×10^{-6} and a return correlation effectively equal to one. Across the top-5 main-family candidates and five rolling validation windows, the rank correlation between AutoQuant and Backtrader replay metrics was 1.000, with maximum monthly-metric difference 3.14×10^{-4} . This supports semantic portability when a custom adapter and explicit ledger are supplied. Freqtrade was also evaluated with its local futures backtester through a programmatic signal adapter derived from the same GP-BO exposure path. The Freqtrade run completed with 65 trades from 265 exposure changes, total trade PnL of 137.87%, and maximum account drawdown of 16.19%. This is reported as a framework-adapter benchmark with explicit adapter semantics.

Table A.29: **Third-party framework replay benchmark.** Backtrader is evaluated through an event-loop adapter over the same STRICT4H executed exposure series and cost/funding profile. Freqtrade is evaluated with its local futures backtester through a programmatic signal adapter derived from the AutoQuant GP-BO exposure path. The table reports an offline framework-replay comparison.

Path	Run status	Backtest tion	devia- tion	Stability check	Scope note
AutoQuant STRICT4H	completed	reference		reference pool	top-5 native per-bar signal, exposure, costs, funding, and net-return ledger
Backtrader adapter	completed	max RMSE 4.68e-06	5.44e-05; rank rho 1.000; max monthly delta 3.14e-04		custom exposure replay with explicit cost/funding ledger
Freqtrade adapter	completed	trade 137.87%; 16.19%	PnL DD	65 trades from 265 exposure changes	local futures backtester with signal adapter

Runtime parity check. Beyond the synthetic and accounting-style checks above, the repository also includes an automated strict- $t + 1$ parity script that compares the rigorous backtest path with the live-style inference wrapper on the same data slice and parameter snapshot. This check was run on three independent SOL/USDT 1h parameter snapshots over 2,000 bars each, with exact agreement in the exported signal and exposure series

(`direction_match_ratio`=1.0, maximum absolute signal difference = 0, maximum absolute exposure difference = 0). This provides code-path consistency evidence for the offline-to-live bridge.

The historical BTC/USDT, ETH/USDT, SOL/USDT, and AVAX/USDT perpetual data used in this study were obtained from major centralized exchanges, primarily Binance, Bybit, and OKX. Due to exchange licensing and storage constraints, the replication package does not redistribute raw tick-level data. The replication materials therefore include scripts and configuration templates that allow qualified researchers to re-download data from original sources, subject to exchange terms, and to regenerate the cleaned 4-hour OHLCV and funding-rate series used in the experiments. Processed summary statistics and anonymized walk-forward tables can be shared with qualified researchers on reasonable request, subject to data-licensing constraints.

High-frequency guard experiments. As part of the broader project, a series of high-frequency ATR-guard experiments on 1m–60m resampled BTC/USDT data were also conducted. These auxiliary experiments are not part of the main empirical results reported in this paper. Empirically, none of the tested configurations delivered a clear improvement in risk-adjusted performance once realistic fees and slippage were taken into account; most settings either left performance essentially unchanged or degraded it by introducing noise-induced turnover. These results are therefore interpreted as context-specific evidence that, in the high-friction cost regime considered here, a 4-hour decision horizon is more suitable than naive 1m–15m overlays, while recognizing that more sophisticated high-frequency designs would require separate study.

Declaration of generative AI and AI-assisted technologies

During the preparation of this manuscript, the author used OpenAI Codex CLI in a limited assistive capacity for English copy-editing and tightening of non-technical prose, and for drafting reproducibility and table/figure traceability checklists. After using this tool, the author reviewed and edited the content as needed and takes full responsibility for the content of the published article. All experimental results, numerical findings, and conclusions reported in this manuscript were generated and verified solely by the author using the described codebase and exported artifacts. This AI tool had no role in producing data, selecting configurations, or computing metrics.

References

- [1] Backtrader contributors, backtrader: Python backtesting library for trading strategies, <https://github.com/mementum/backtrader>, computer software. Accessed 2025-12-24 (2025).
- [2] Zipline contributors, Zipline: A pythonic algorithmic trading library, <https://github.com/zipline-live/zipline-reloaded>, computer software. Accessed 2025-12-24 (2025).
- [3] Freqtrade contributors, Freqtrade: Free, open source crypto trading bot, <https://github.com/freqtrade/freqtrade>, computer software. Accessed 2025-12-24 (2025).
- [4] AI4Finance-Foundation, Finrl: Deep reinforcement learning framework for quantitative finance, <https://github.com/AI4Finance-Foundation/FinRL>, computer software. Accessed 2025-12-24 (2025).
- [5] J. Bergstra, R. Bardenet, Y. Bengio, B. K. 'egl, Algorithms for hyper-parameter optimization, in: Advances in Neural Information Processing Systems, Vol. 24, 2011, pp. 2546–2554, accessed 2025-12-24.
URL https://papers.nips.cc/paper_files/paper/2011/hash/86e8f7ab32cfd12577bc2619bc635690-Abstract.html
- [6] T. Akiba, S. Sano, T. Yanase, T. Ohta, M. Koyama, Optuna: A next-generation hyperparameter optimization framework, in: Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, 2019, pp. 2623–2631, accessed 2025-12-24. doi:10.1145/3292500.3330701.
URL <https://doi.org/10.1145/3292500.3330701>
- [7] J. Bergstra, Y. Bengio, Random search for hyper-parameter optimization, Journal of Machine Learning Research 13 (2012) 281–305, accessed 2025-12-24.
URL <https://jmlr.org/papers/v13/bergstra12a.html>
- [8] T. Philippon, The fintech opportunity, Tech. Rep. w22476, National Bureau of Economic Research (2016). doi:10.3386/w22476.

- [9] P. Gomber, J.-A. Koch, M. Siering, Digital finance and fintech: current research and future research directions, *Journal of Business Economics* 87 (5) (2017) 537–580. doi:10.1007/s11573-017-0852-x.
- [10] F. Hutter, L. Kotthoff, J. Vanschoren, *Automated Machine Learning: Methods, Systems, Challenges*, Springer, Cham, 2019, accessed 2025-12-24. doi:10.1007/978-3-030-05318-5.
URL <https://doi.org/10.1007/978-3-030-05318-5>
- [11] M. López de Prado, *Advances in Financial Machine Learning*, John Wiley & Sons, Hoboken, NJ, 2018, accessed 2025-12-24.
URL <https://openlibrary.org/books/OL33645013M>
- [12] Y. Liu, A. Tsyvinski, Risks and returns of cryptocurrency, *Review of Financial Studies* 34 (6) (2020) 2689–2727. doi:10.1093/rfs/hhaa113.
- [13] Y. Liu, A. Tsyvinski, X. Wu, Common risk factors in cryptocurrency, *The Journal of Finance* 77 (2) (2022) 1133–1177. doi:10.1111/jofi.13119.
- [14] I. Makarov, A. Schoar, Trading and arbitrage in cryptocurrency markets, *Journal of Financial Economics* 135 (2) (2020) 293–319, accessed 2025-12-24. doi:10.1016/j.jfineco.2019.07.001.
URL <https://doi.org/10.1016/j.jfineco.2019.07.001>
- [15] D. Ackerer, J. Hugonnier, U. Jermann, Perpetual futures pricing, Tech. Rep. w32936, National Bureau of Economic Research (2024). doi:10.3386/w32936.
- [16] W. Gornall, J. M. Rinaldi, Y. Xiao, Perpetual futures and basis risk: Evidence from cryptocurrency, *SSRN Electronic Journal* (2025). doi:10.2139/ssrn.5036933.
- [17] Q. Ruan, A. Streltsov, Perpetual futures contracts and cryptocurrency market quality, *SSRN Electronic Journal* (2025). doi:10.2139/ssrn.4218907.
- [18] E. Chen, M. Ma, Z. Nie, Perpetual future contracts in centralized and decentralized exchanges: Mechanism and traders’ behavior, *Electronic Markets* 34 (1) (2024) 35. doi:10.1007/s12525-024-00715-1.

- [19] J.-C. Hung, H.-C. Liu, J. J. Yang, Trading activity and price discovery in bitcoin futures markets, *Journal of Empirical Finance* 62 (2021) 107–120. doi:10.1016/j.jempfin.2021.03.001.
- [20] A. Shynkevich, Impact of bitcoin futures on the informational efficiency of bitcoin spot market, *Journal of Futures Markets* 41 (1) (2020) 115–134. doi:10.1002/fut.22164.
- [21] A. Malik, A comparison of machine learning and econometric models for pricing perpetual bitcoin futures and their application to algorithmic trading, *Expert Systems* 40 (10) (2023). doi:10.1111/exsy.13414.
- [22] M. López de Prado, Seven sins of quantitative investing, *The Journal of Portfolio Management* 45 (1) (2019) 12–25.
- [23] J. Snoek, H. Larochelle, R. P. Adams, Practical bayesian optimization of machine learning algorithms, in: F. Pereira, C. J. C. Burges, L. Bottou, K. Q. Weinberger (Eds.), *Advances in Neural Information Processing Systems*, Vol. 25, Curran Associates, Inc., 2012, pp. 2951–2959, accessed 2025-12-24.
URL https://papers.nips.cc/paper_files/paper/2012/hash/05311655a15b75fab86956663e1819cd-Abstract.html
- [24] D. E. Goldberg, *Genetic Algorithms in Search, Optimization, and Machine Learning*, Addison-Wesley, Reading, MA, 1989.
- [25] R. Storn, K. Price, Differential evolution – a simple and efficient heuristic for global optimization over continuous spaces, *Journal of Global Optimization* 11 (4) (1997) 341–359, accessed 2025-12-24. doi:10.1023/A:1008202821328.
URL <https://doi.org/10.1023/A:1008202821328>
- [26] N. Hansen, The cma evolution strategy: A comparing review, in: *Towards a New Evolutionary Computation*, Springer, 2006, pp. 75–102, accessed 2025-12-24. doi:10.1007/3-540-32494-1_4.
URL https://doi.org/10.1007/3-540-32494-1_4
- [27] R. Sullivan, A. Timmermann, H. White, Data-snooping, technical trading rule performance, and the bootstrap, *The Journal of Finance* 54 (5) (1999) 1647–1691. doi:10.1111/0022-1082.00163.

- [28] H. White, A reality check for data snooping, *Econometrica* 68 (5) (2000) 1097–1126. doi:10.1111/1468-0262.00152.
- [29] P. R. Hansen, A test for superior predictive ability, *Journal of Business & Economic Statistics* 23 (4) (2005) 365–380. doi:10.1198/073500105000000063.
- [30] C. R. Harvey, Y. Liu, H. Zhu, ... and the cross-section of expected returns, *Review of Financial Studies* 29 (1) (2015) 5–68. doi:10.1093/rfs/hhv059.
- [31] D. H. Bailey, M. López de Prado, The deflated sharpe ratio: Correcting for selection bias, backtest overfitting, and non-normality, *The Journal of Portfolio Management* 40 (5) (2014) 94–107, accessed 2025-12-24. doi:10.3905/jpm.2014.40.5.094.
URL <https://doi.org/10.3905/jpm.2014.40.5.094>
- [32] D. H. Bailey, J. M. Borwein, M. López de Prado, Q. J. Zhu, The probability of backtest overfitting, *The Journal of Computational Finance* 20 (4) (2017) 39–69, accessed 2025-12-24. doi:10.21314/jcf.2016.322.
URL <https://doi.org/10.21314/jcf.2016.322>
- [33] F. Schär, Decentralized finance: On blockchain- and smart contract-based financial markets, *Federal Reserve Bank of St. Louis Review* 103 (2) (2021) 153–174, accessed 2025-12-24. doi:10.20955/r.103.153-74.
URL <https://doi.org/10.20955/r.103.153-74>
- [34] H. R. Künsch, The jackknife and the bootstrap for general stationary observations, *The Annals of Statistics* 17 (3) (1989) 1217–1241, accessed 2025-12-24. doi:10.1214/aos/1176347265.
URL <https://doi.org/10.1214/aos/1176347265>